Acknowledgements & Copyright Notice

© 2026 Dr. Pandarasamy Arjunan, Indian Institute of Science, Bangalore. All rights reserved.

These slides are distributed under a Creative Commons License. See Creative Commons Attribution-ShareAlike 2.0 License.

The author makes these slides available for educational purposes. They may not be used or distributed for commercial purposes. You may copy, use, or distribute these slides for educational purposes, provided you cite the author as the source.

Credits: This presentation includes slides and images from various books, research articles, and online sources, including:

- NVIDIA's Edge AI and Robotics Teaching Kit
- Harvard - TinyML Courseware and Machine Learning Systems
- MIT - TinyML and Efficient Deep Learning Computing
- Dive into Deep Learning and Luis Serrano's Academy and his books and videos
- Hands-on Machine Learning with Scikit-Learn, Keras and TensorFlow book
- IoT Fundamentals: Networking Technologies, Protocols, and Use Cases for the Internet of Things, 2017 by David Hanes
- Introduction to IoT, 2021 by Anandarup Mukherjee; *AI at the Edge* by Daniel Situnayake, Jenny Plunkett - O'Reilly

The author acknowledges and thanks the creators for their contributions. Images and content are used under fair use for educational purposes.

Disclaimer: All logos, images, and other trademarks are the property of their respective owners. If you believe that any content in this presentation violates copyright laws, please contact samy@iisc.ac.in to have it promptly removed or appropriately attributed.

Instruction Enhancement Program (IEP) on

Edge AI

Lecture 5: Model Optimization Techniques

Pandarasamy Arjunan (Samy)
Assistant Professor
Robert Bosch Centre for Cyber Physical Systems
Indian Institute of Science, Bengaluru
samy@iisc.ac.in

Image source: https://leverx.com

Lecture Plan

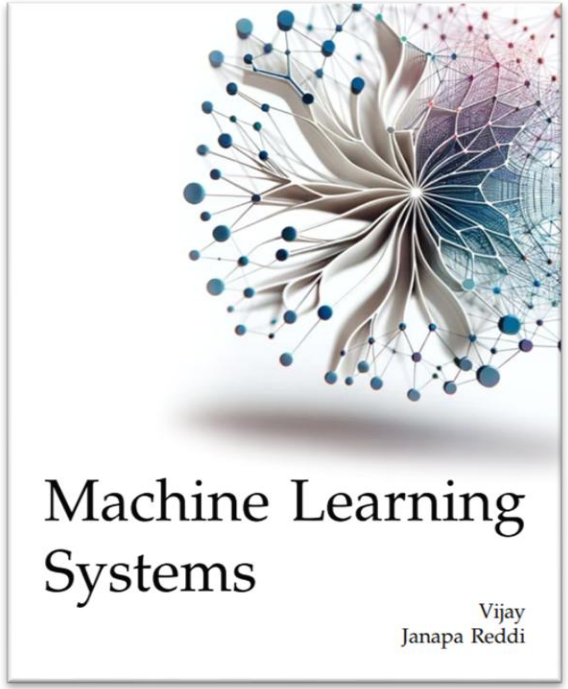
Today

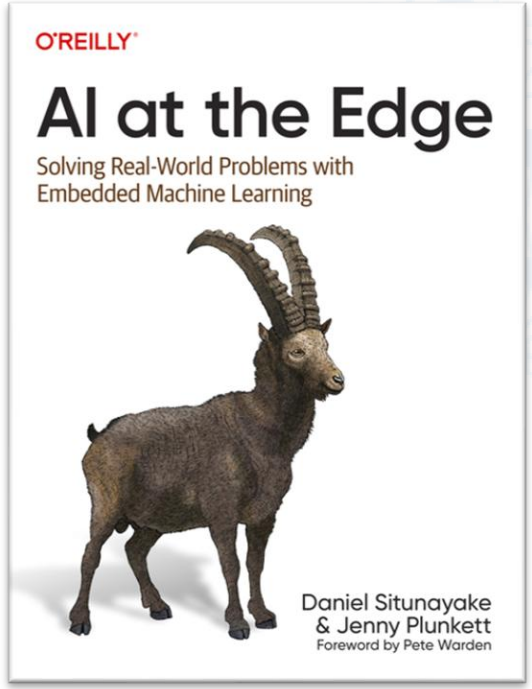
- **Lecture 1**: Introduction to IoT, Edge Analytics, and Edge AI
- **Lecture 2**: Tiny Machine Learning (TinyML)
- **Lab 1**: Human Activity Recognition using TinyML

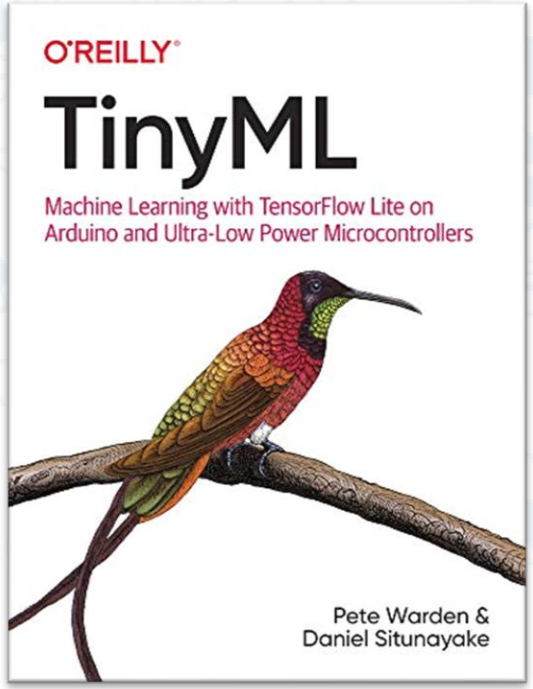
Tomorrow

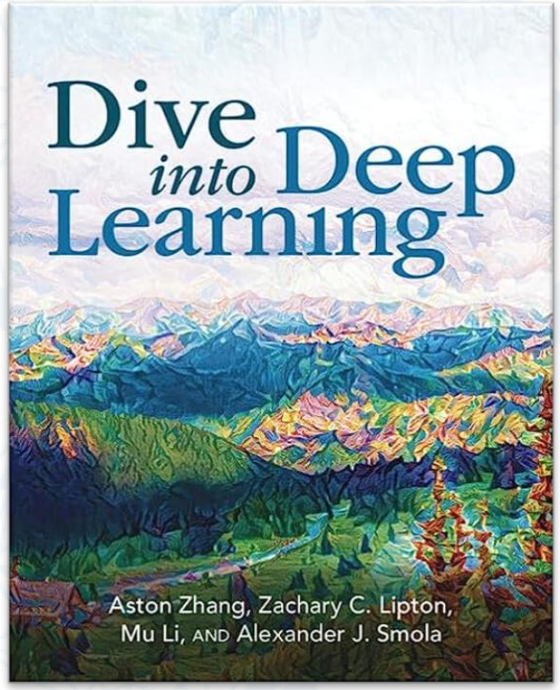
- **Lecture 3**: Overview of Embedded Computer Vision
- **Lecture 4**: Efficient Deep Learning and Efficiency Metrics
- **Lecture 5: Model Optimization Techniques (Quantization, Pruning, etc.)**
- **Lab 2**: Embedded Computer Vision
- **Lab 3**: Model Optimization for Edge AI (online)

Books and reference materials





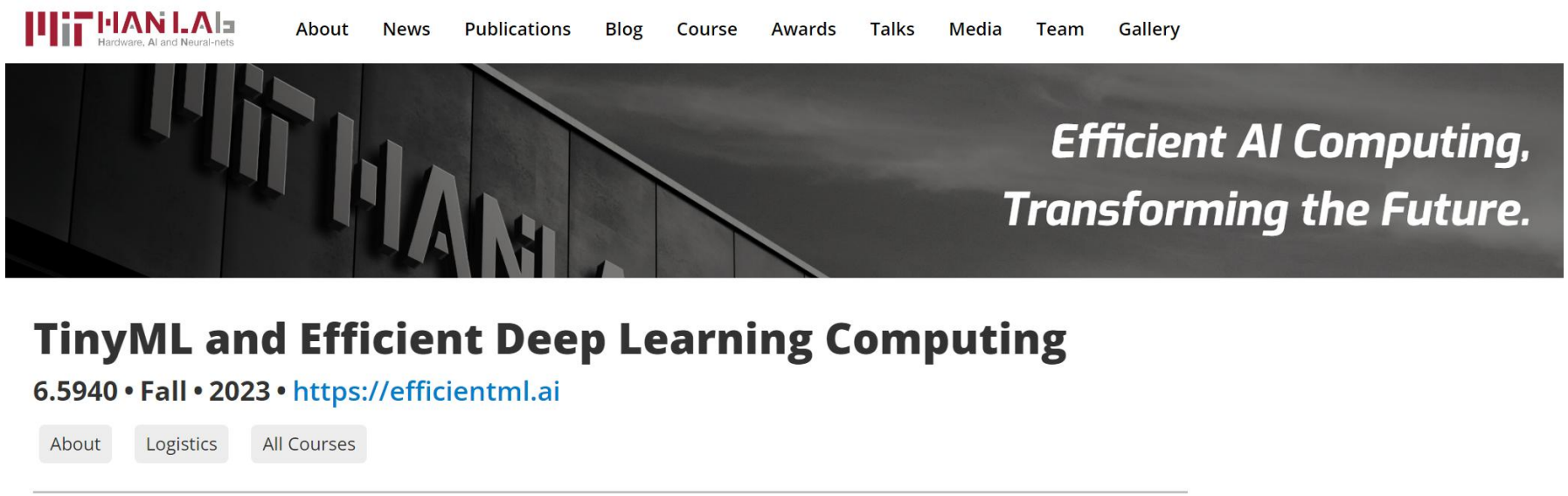




Plus, hands-on tutorials and research papers

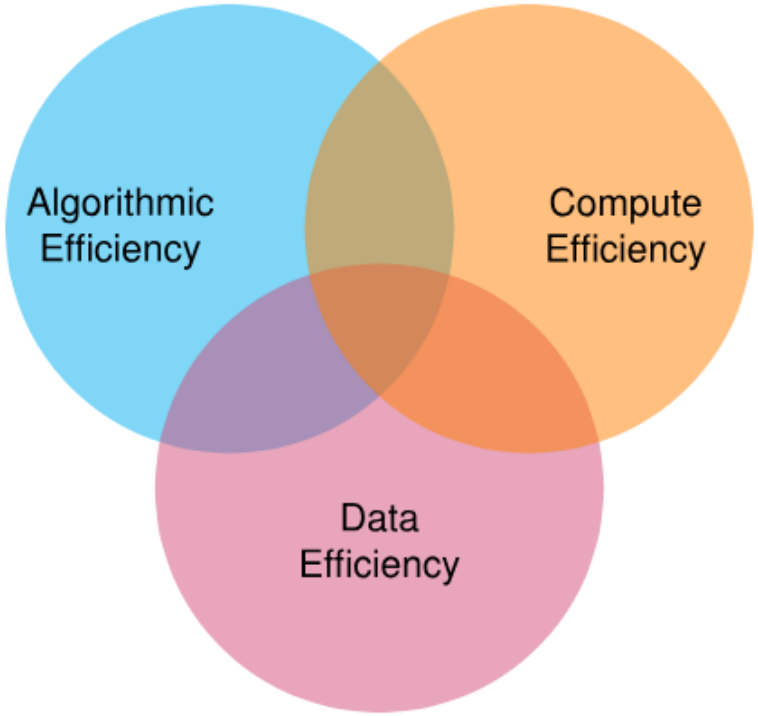
Reference

TinyML and Efficient Deep Learning Computing - https://hanlab.mit.edu/courses/2023-fall-65940

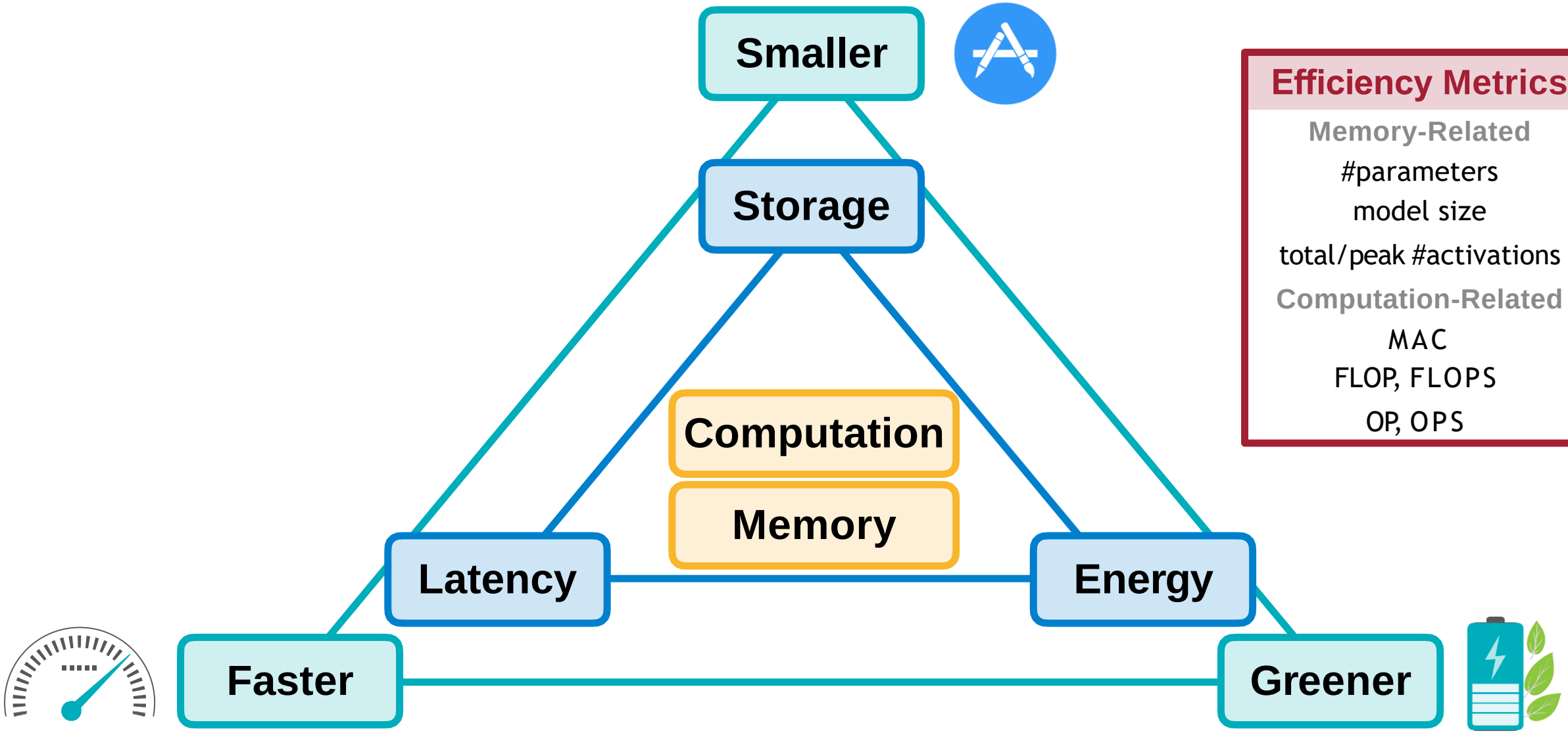


Efficient AI

- **Algorithmic Efficiency** focuses on the model architecture.
- **Compute Efficiency** addresses hardware utilization
- **Data Efficiency** shapes how the model learns



Efficiency Metrics



Efficiency Metrics

Memory-Related

#parameters

model size

total/peak #activations

Computation-Related

MAC

FLOP, FLOPS

OP, OPS

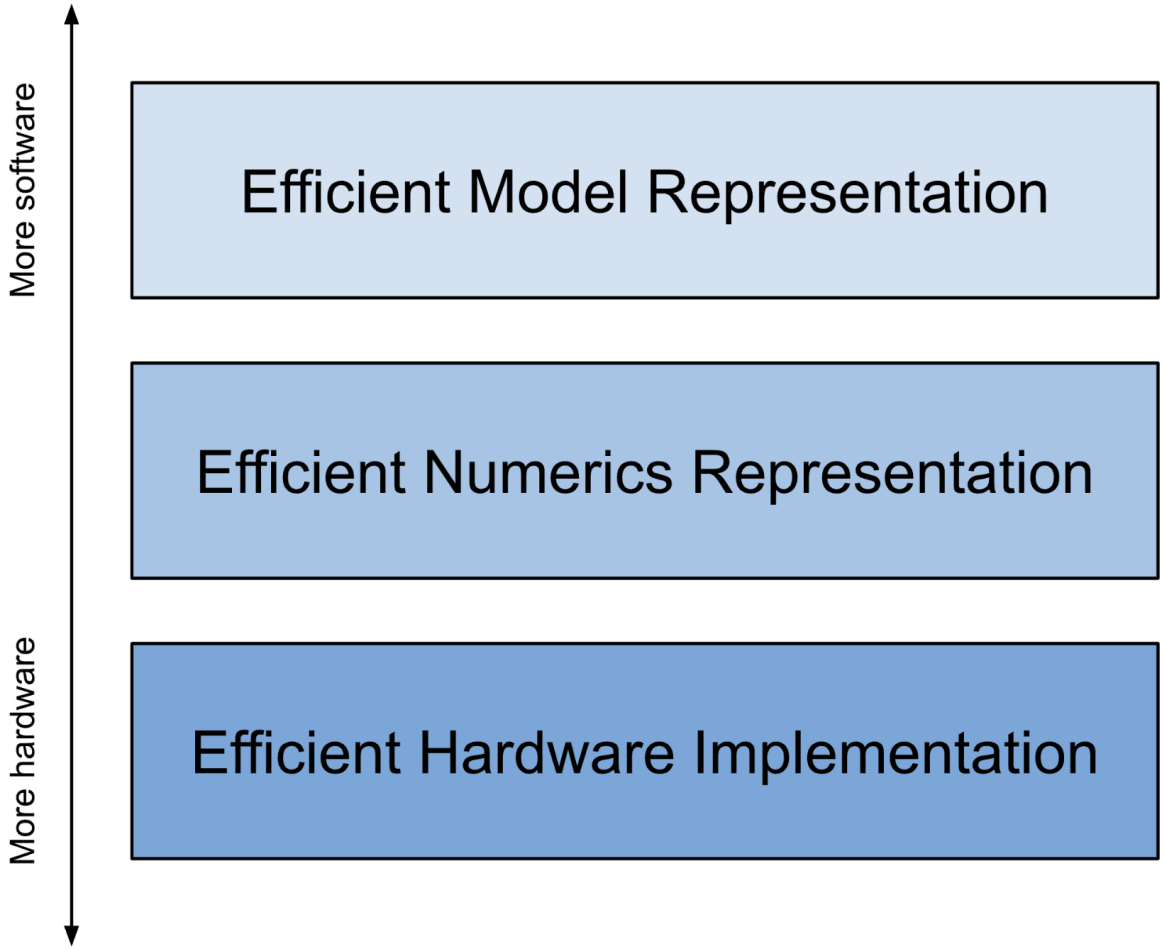
Image source: 1

Model Optimization Framework

"Systematic transformation of AI models to maximize computational efficiency while preserving task performance, enabling deployment across diverse hardware constraints"

System Constraint	Model Representation	Numerical Precision	Architectural Efficiency
Computational Cost	✗	✓	✓
Memory and Storage	✓	✓	✗
Latency and Throughput	✓	✗	✓
Energy Efficiency	✗	✓	✓
Scalability	✓	✗	✓

Model Optimization Framework

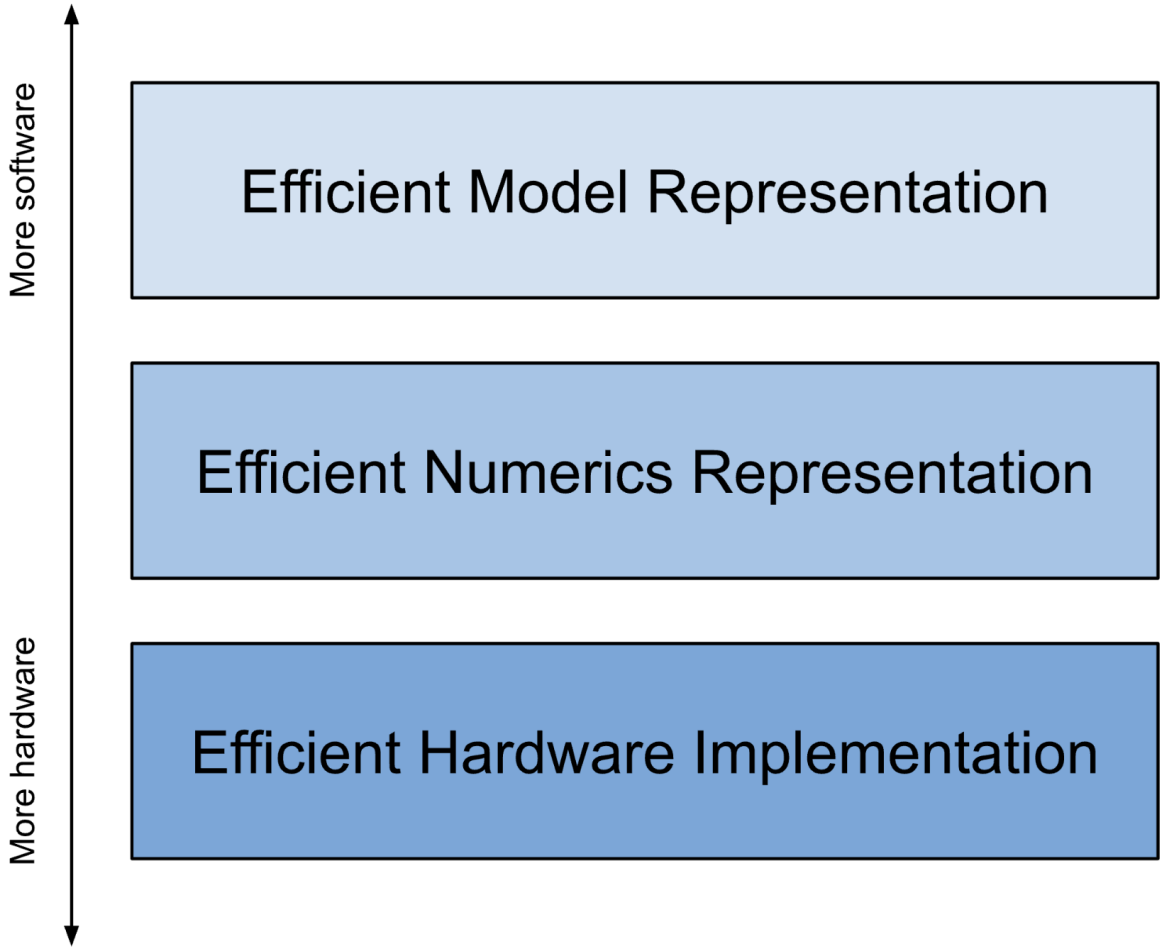


- Pruning & Sparsity
- Knowledge Distillation
- Neural Architecture Search

- Quantization

- Optimize execution

Model Optimization Framework



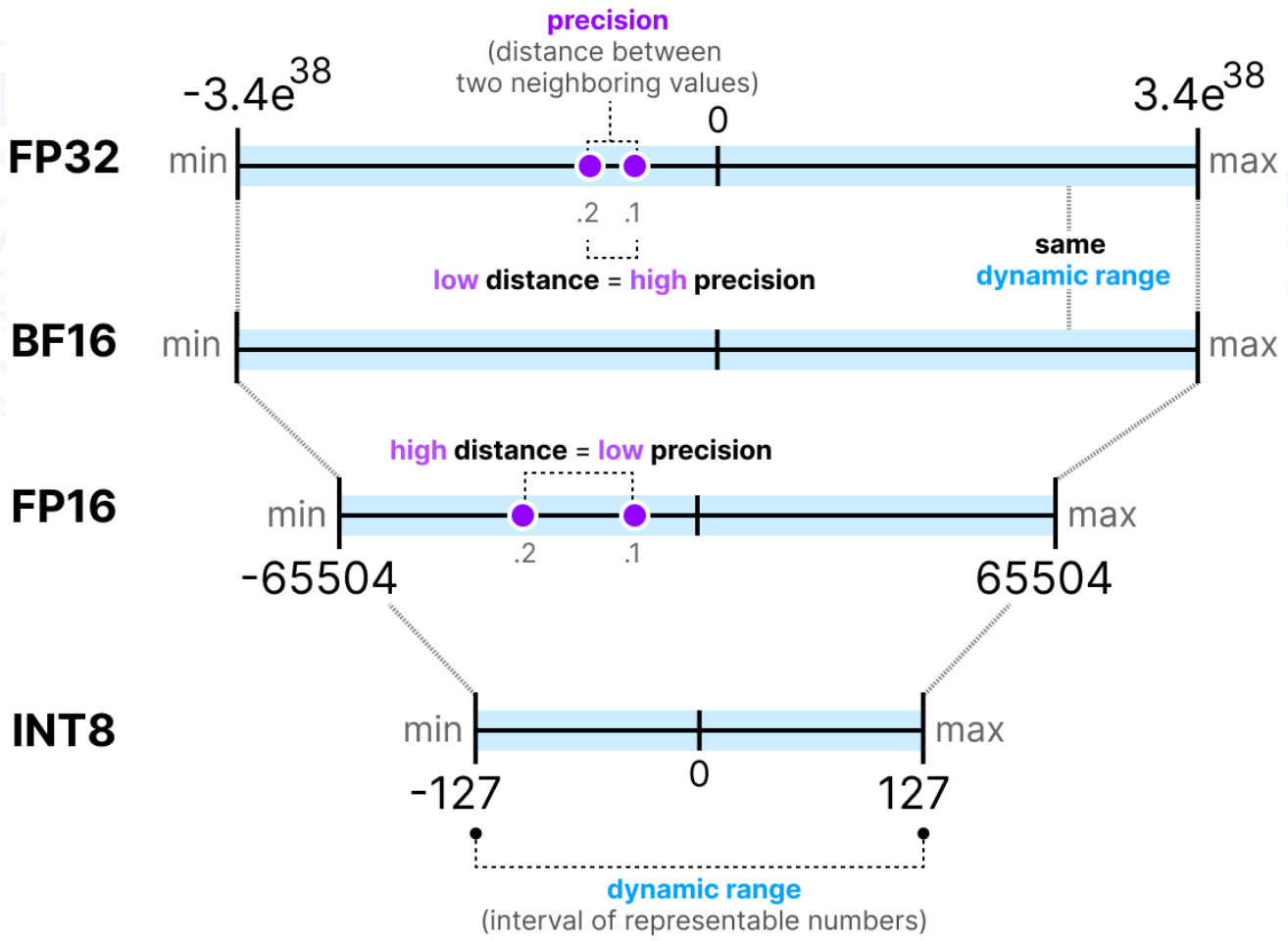
- Pruning & Sparsity
- Knowledge Distillation
- Neural Architecture Search

- Quantization

- Optimize execution

How are numbers represented in computers?

- **INT**: whole numbers represented in binary without a fractional component.
  - 4/8/16/32/64 bits
- **FLOAT**: real numbers represented using a sign, exponent, and mantissa to encode both large and small values with fractional precision
  - 8/16/32/64/128 bits

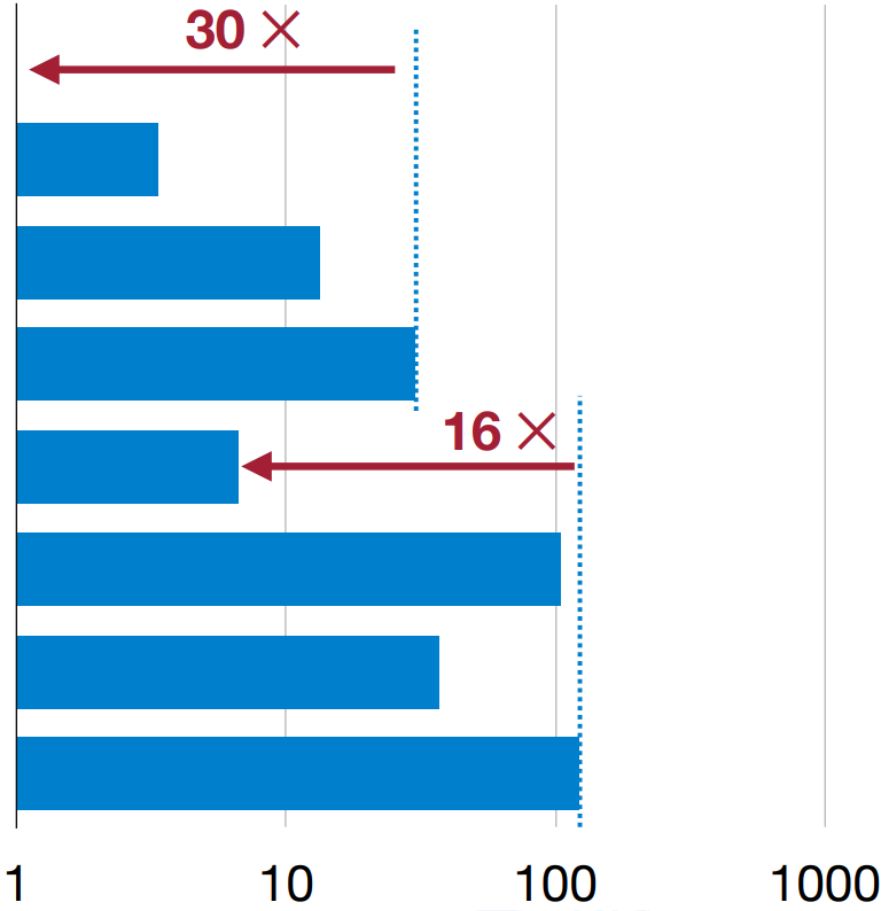


Source: https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization

Low Bit Operations are Cheaper

Operation	Energy [pJ]
8 bit int ADD	0.03
32 bit int ADD	0.1
16 bit float ADD	0.4
32 bit float ADD	0.9
8 bit int MULT	0.2
32 bit int MULT	3.1
16 bit float MULT	1.1
32 bit float MULT	3.7

Rough Energy Cost For Various Operations in 45nm 0.9V

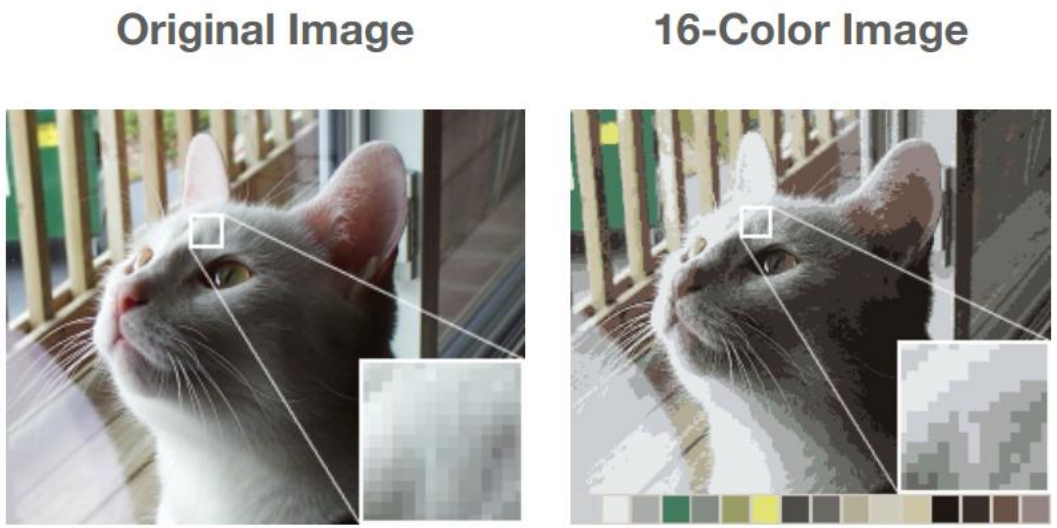


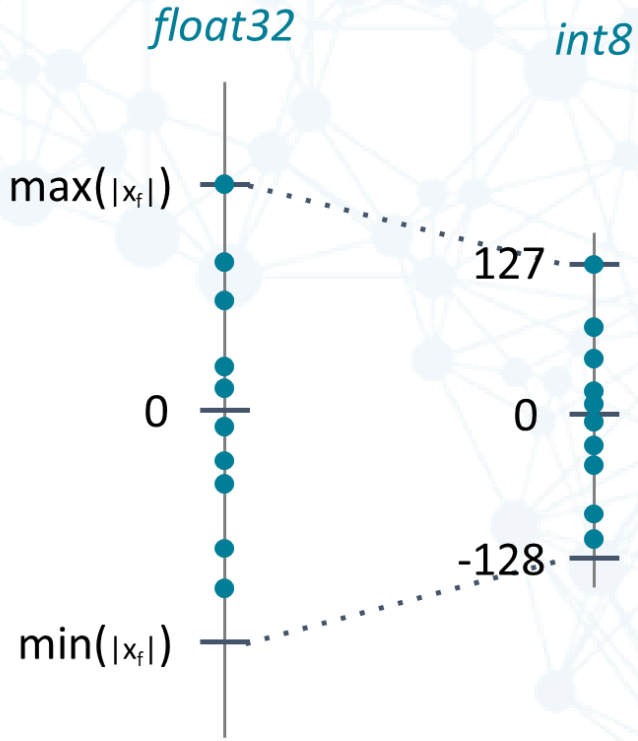
Comparison of Numerical Precision Formats

Precision Format	Bit-Width	Storage Reduction (vs FP32)	Compute Speed (vs FP32)	Power Consumption	Use Cases
FP32 (Single-Precision)	32-bit	Baseline (1×)	Baseline (1×)	High	Training & inference (general-purpose)
FP16 (Half-Precision)	16-bit	2× smaller	2× faster on FP16-optimized hardware	Lower	Accelerated training, inference (NVIDIA Tensor Cores, TPUs)
bfloat16 (Brain Floating Point)	16-bit	2× smaller	Similar speed to FP16, better dynamic range	Lower	Training on TPUs, transformer-based models
TF32 (TensorFloat-32)	19-bit	Similar to FP16	Up to 8× faster on NVIDIA Ampere GPUs	Lower	Training on NVIDIA GPUs
FP8 (Floating-Point 8-bit)	8-bit	4× smaller	Faster than INT8 in some cases	Significantly lower	Efficient training/inference (H100, AI accelerators)
INT8 (8-bit Integer)	8-bit	4× smaller	4–8× faster than FP32	Significantly lower	Quantized inference (Edge AI, mobile AI, NPUs)
INT4 (4-bit Integer)	4-bit	8× smaller	Hardware-dependent	Extremely low	Ultra-low-power AI, experimental quantization
Binary/Ternary (1-bit / 2-bit)	1–2-bit	16–32× smaller	Highly hardware-dependent	Lowest	Extreme efficiency (binary/ternary neural networks)

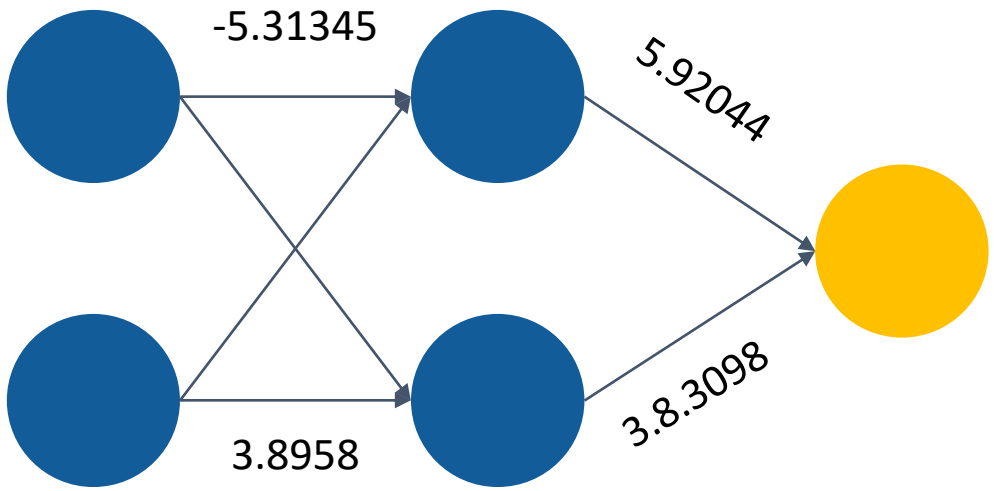
What is Quantization?

Quantization is an optimization that works by **reducing the precision of the numbers** used to represent a model's parameters, which by default are 32-bit floating point numbers. This results in a **smaller model size, better portability** and **faster computation**.

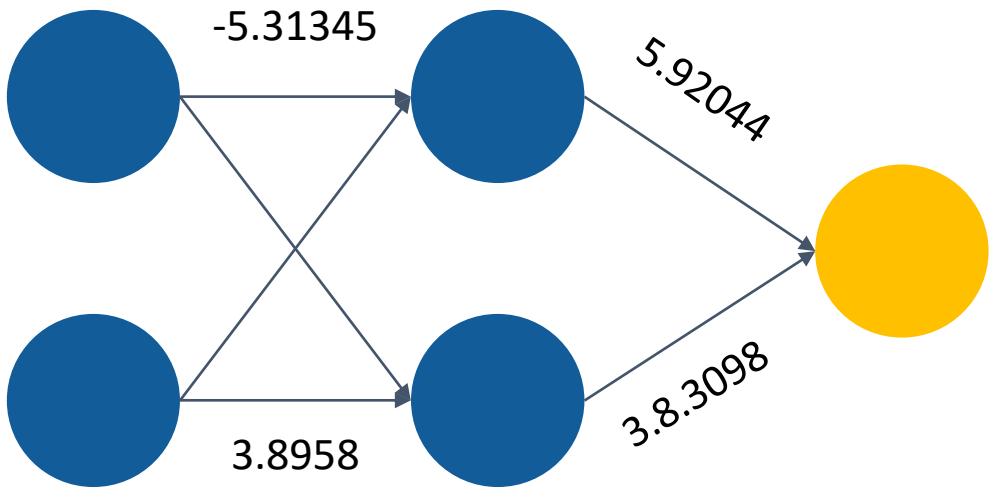


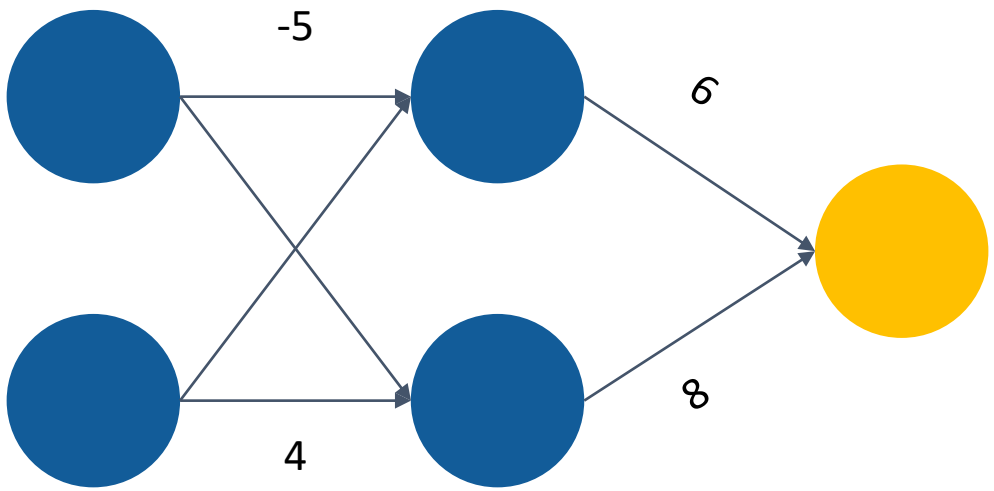


What is Quantization?

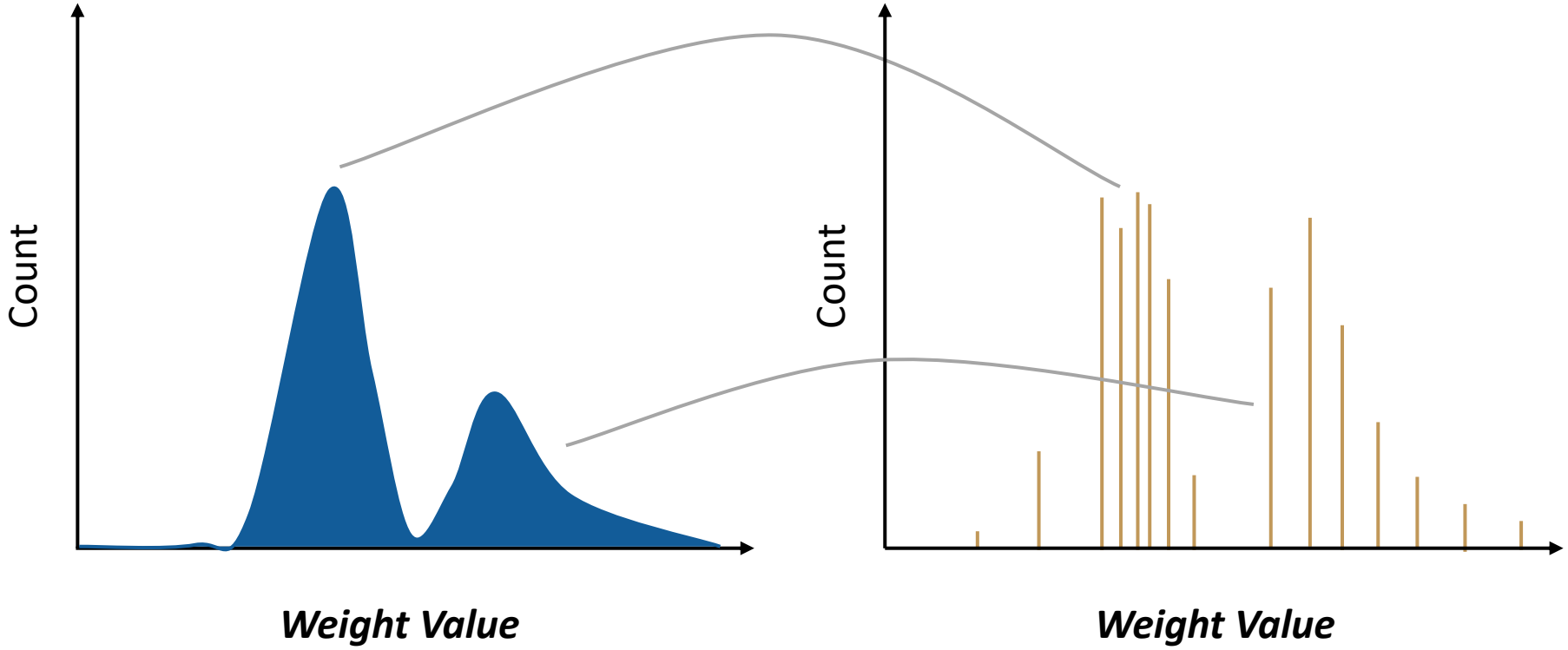


What is Quantization?

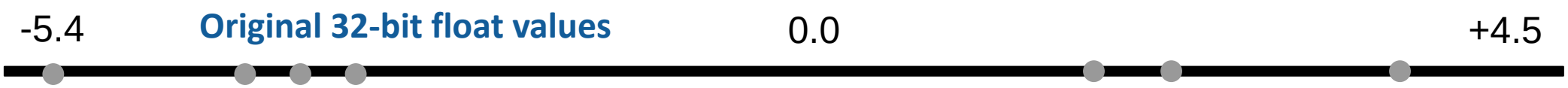




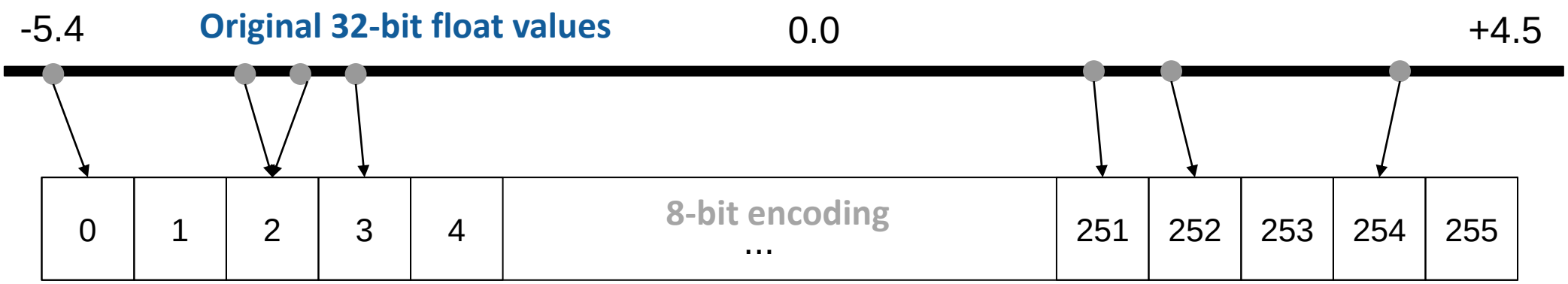
Reduce Precision (Discretize)



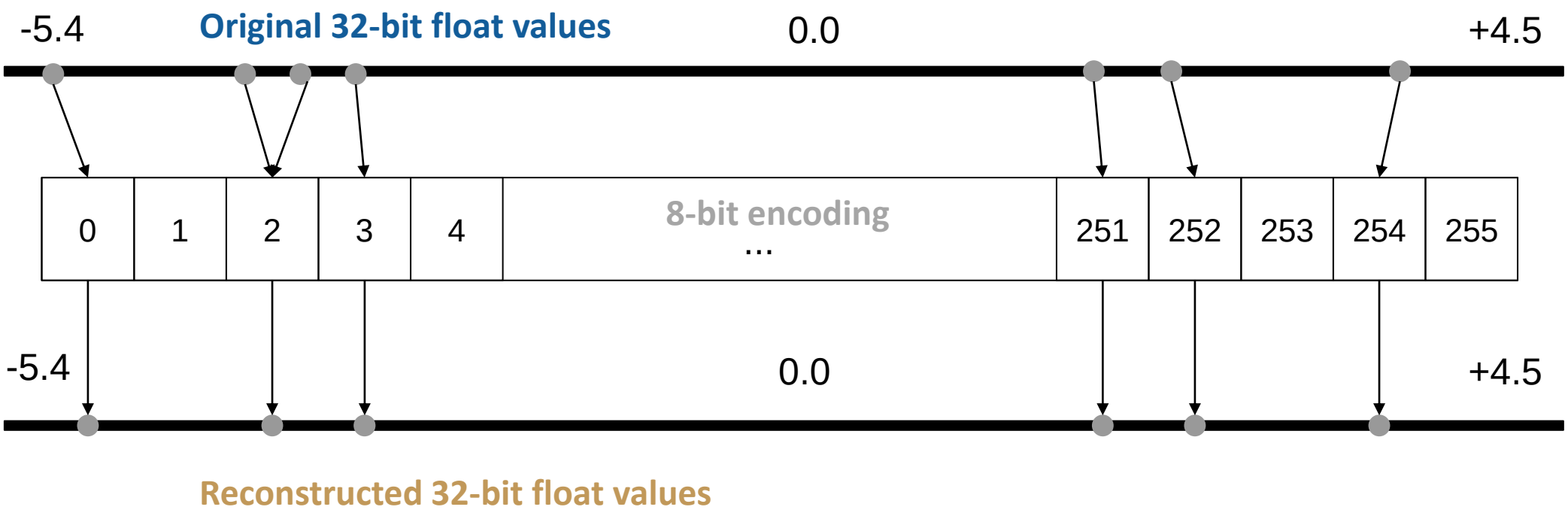
Reduce Precision (Discretize)



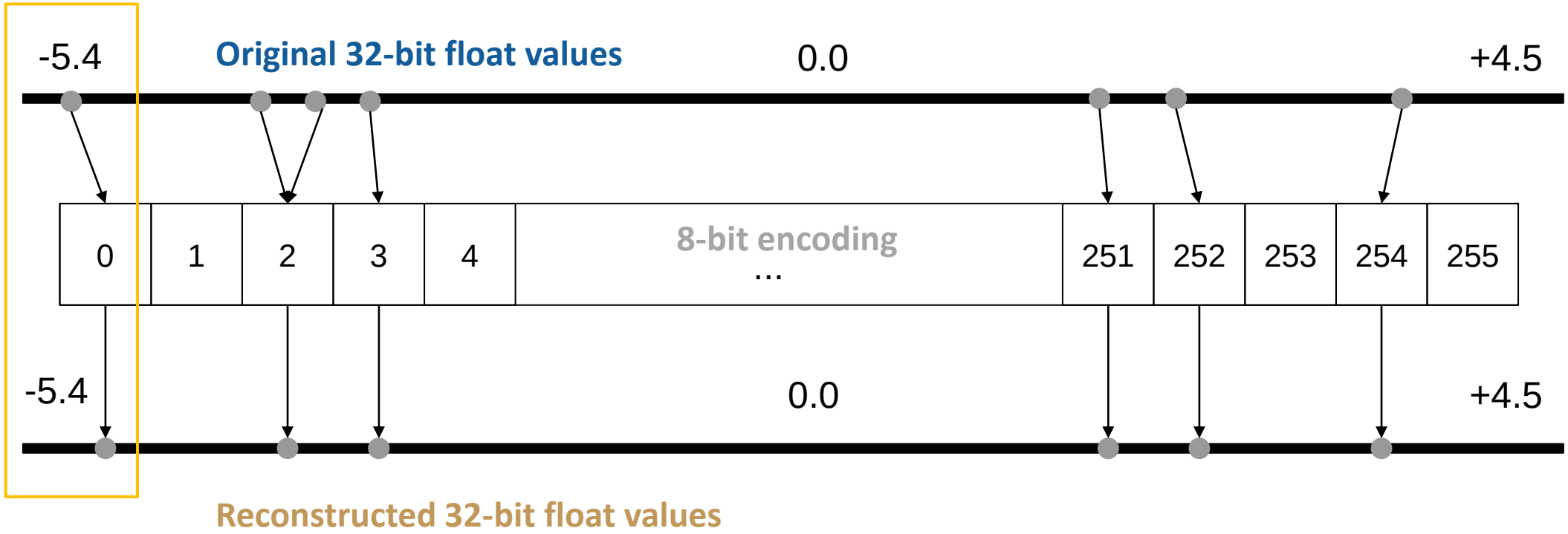
Reduce Precision (Discretize)



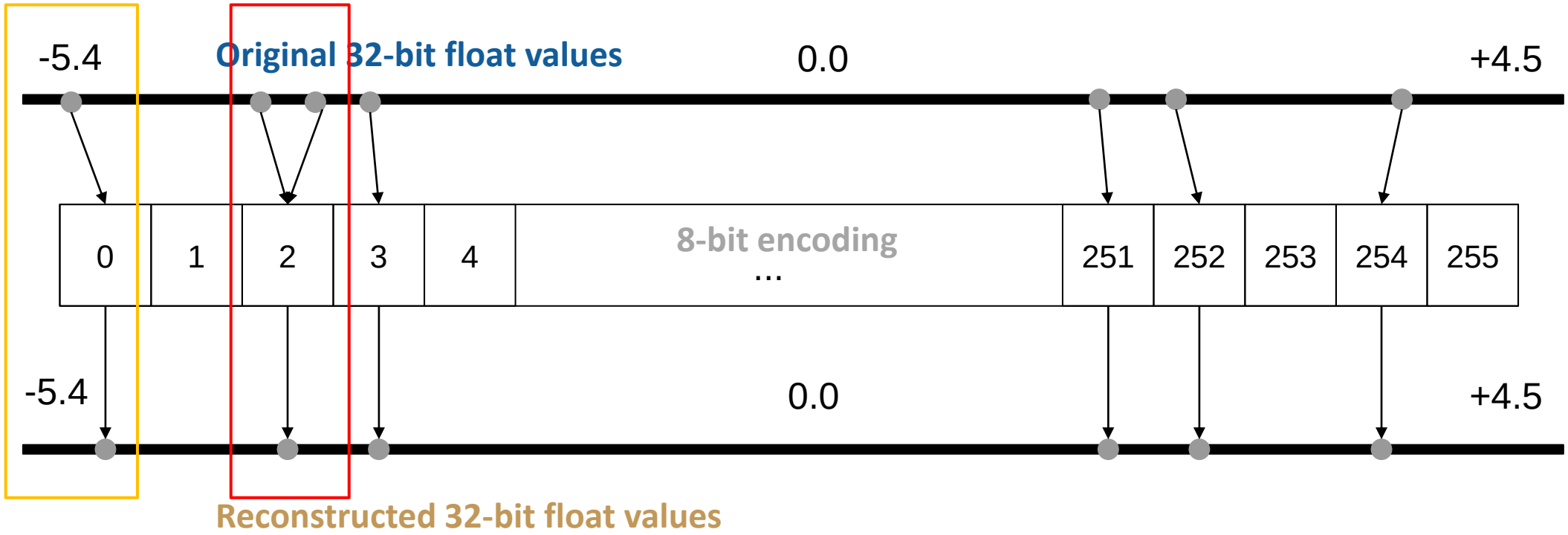
Reduce Precision (Discretize)



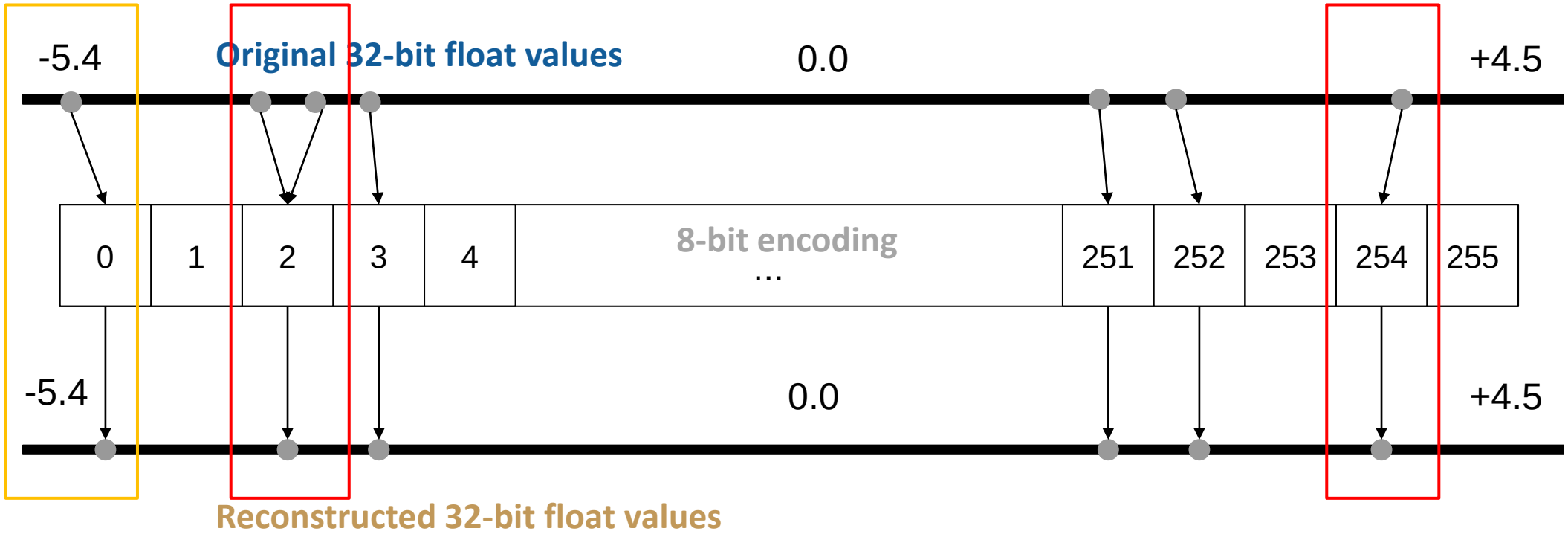
Reduce Precision (Discretize)



Reduce Precision (Discretize)



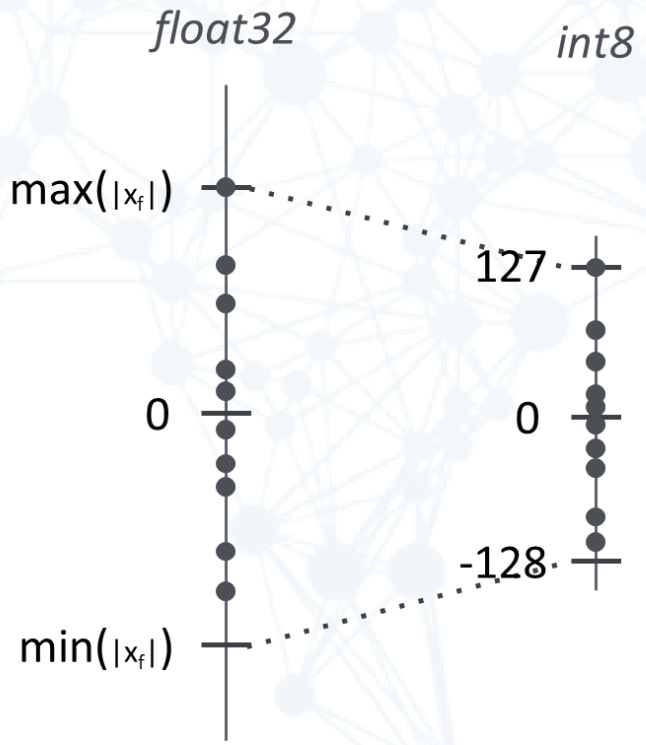
Reduce Precision (Discretize)

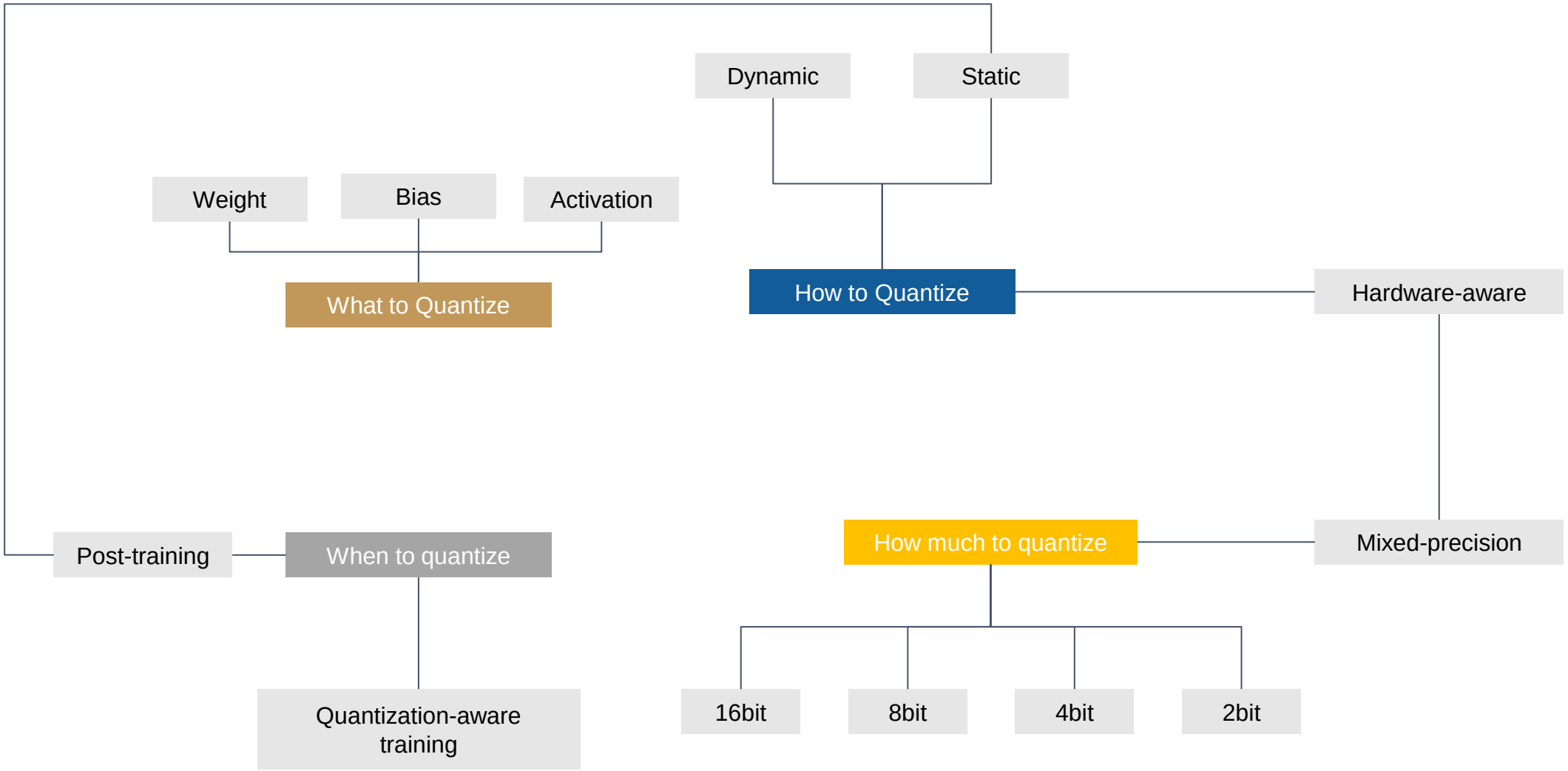


What is Quantization?

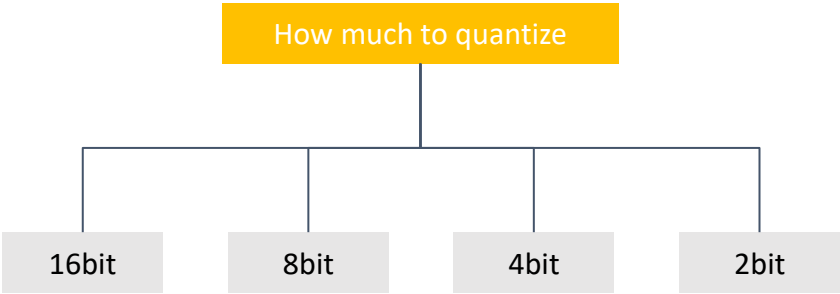
Quantization is an optimization that works by **reducing the precision** of the numbers used to represent a model's parameters, which by default are 32-bit floating point numbers. This results in a **smaller model size**, **better portability** and **faster computation**.

- Why do we quantize?
- How much to quantize?
- When to quantize?
- What to quantize?
- How to quantize?

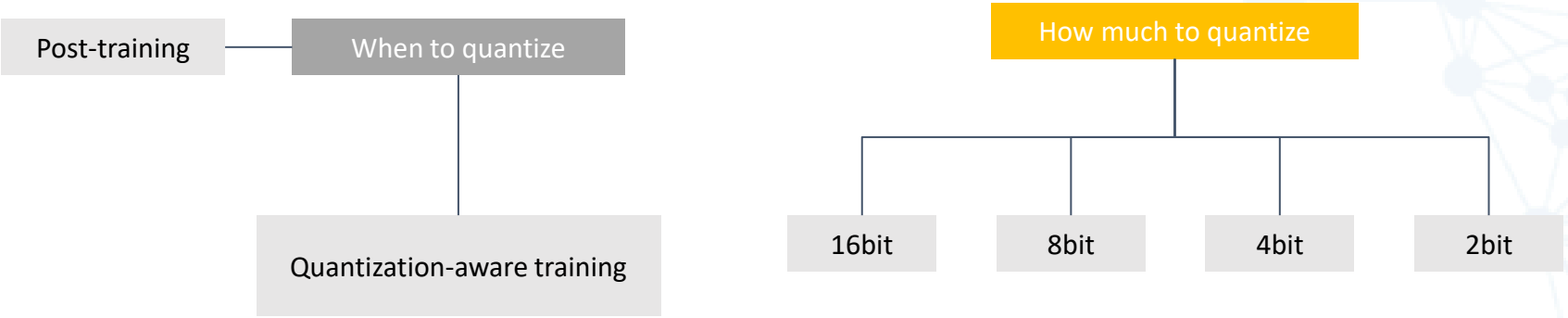



Dynamic
Static
Weight
Bias
Activation
What to Quantize
How to Quantize
Hardware-aware
Post-training
When to quantize
How much to quantize
Mixed-precision
Quantization-aware training
16bit
8bit
4bit
2bit

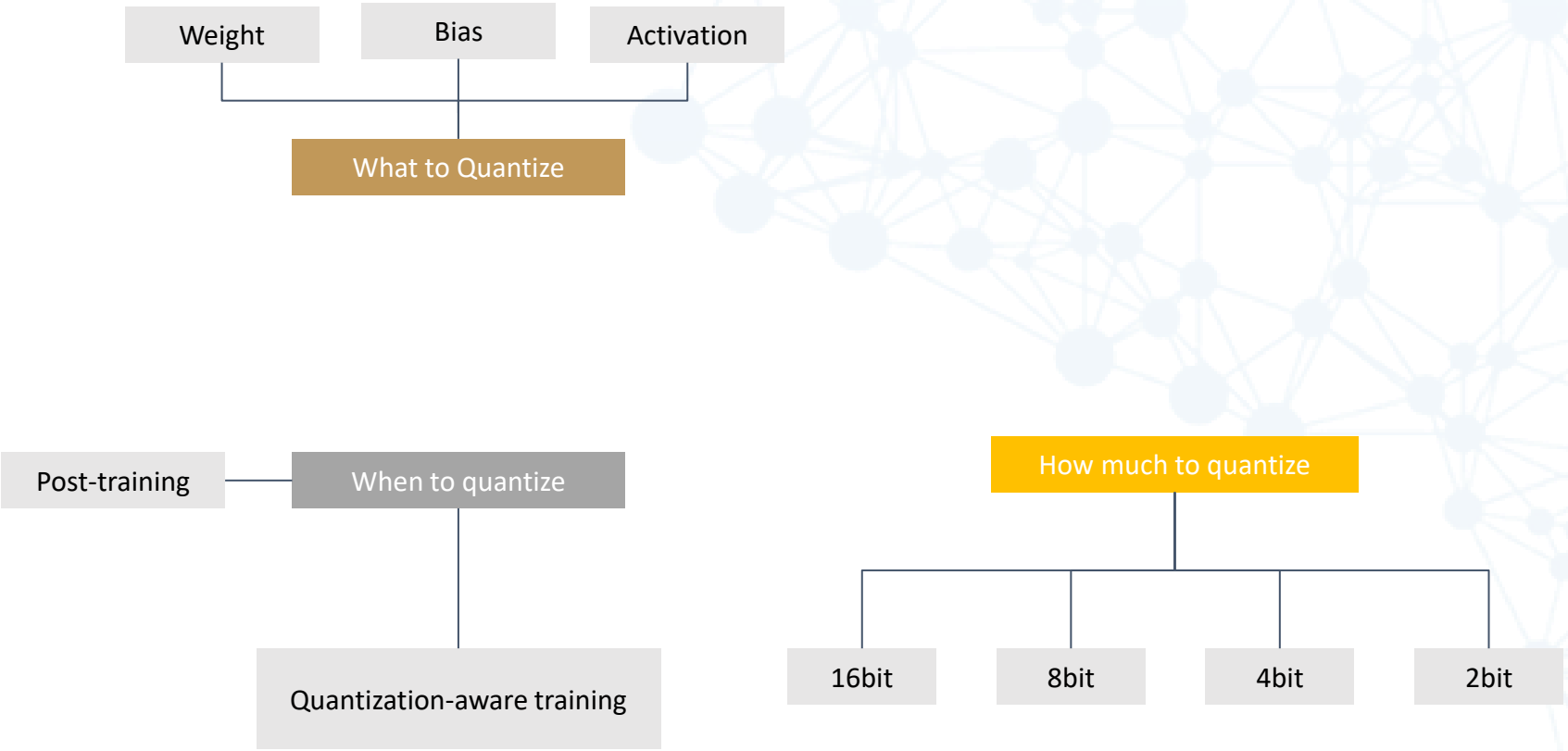
Quantization



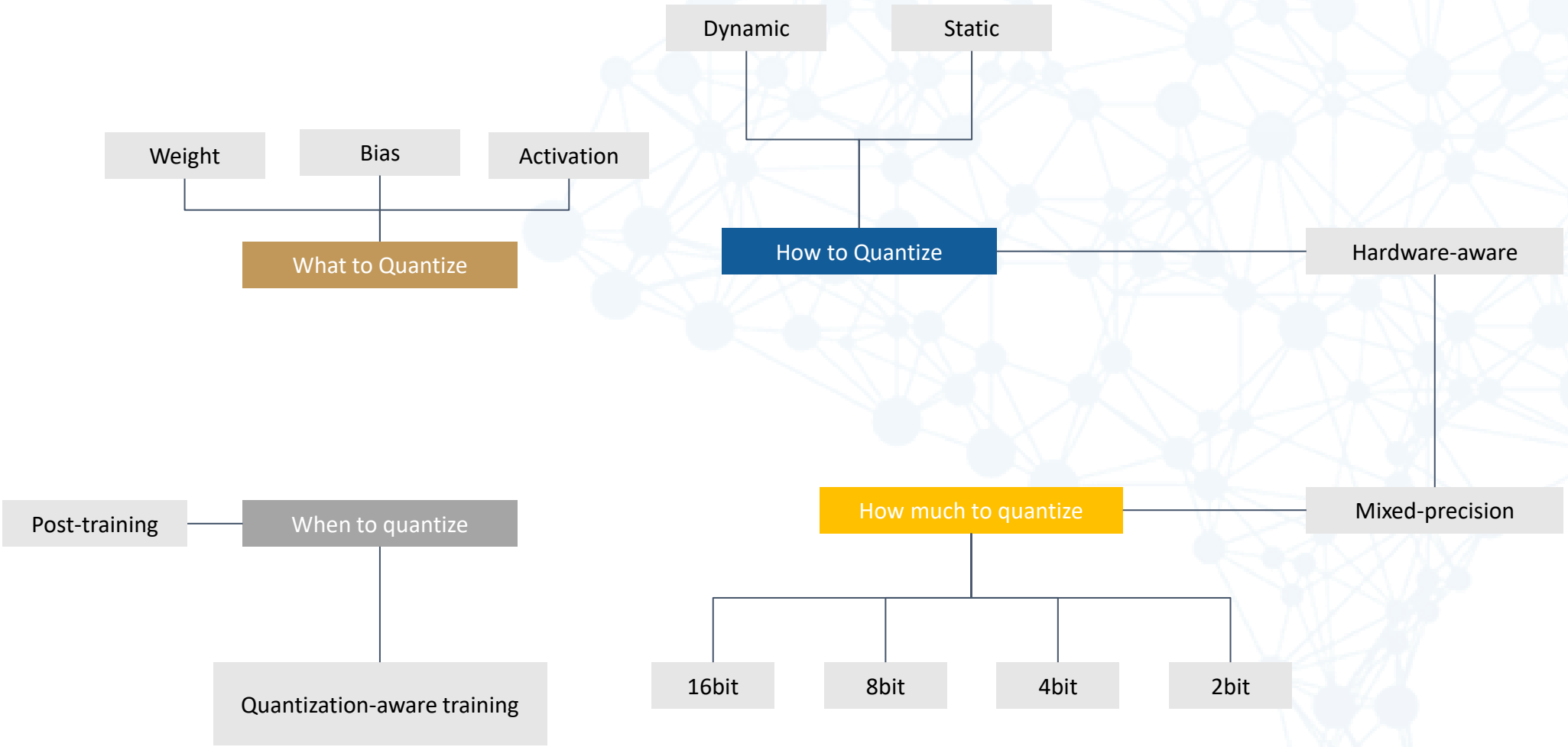
Quantization



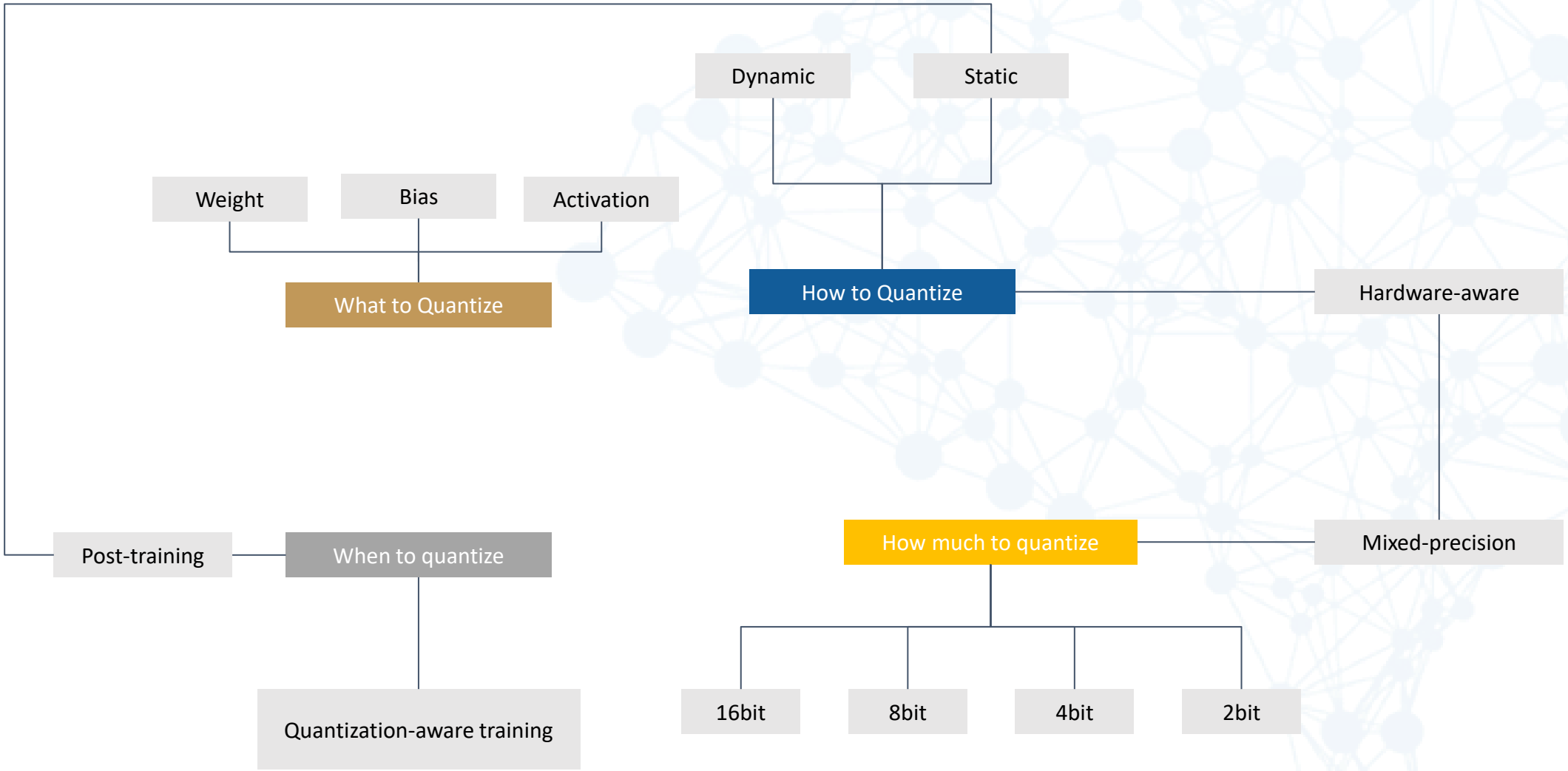
Quantization



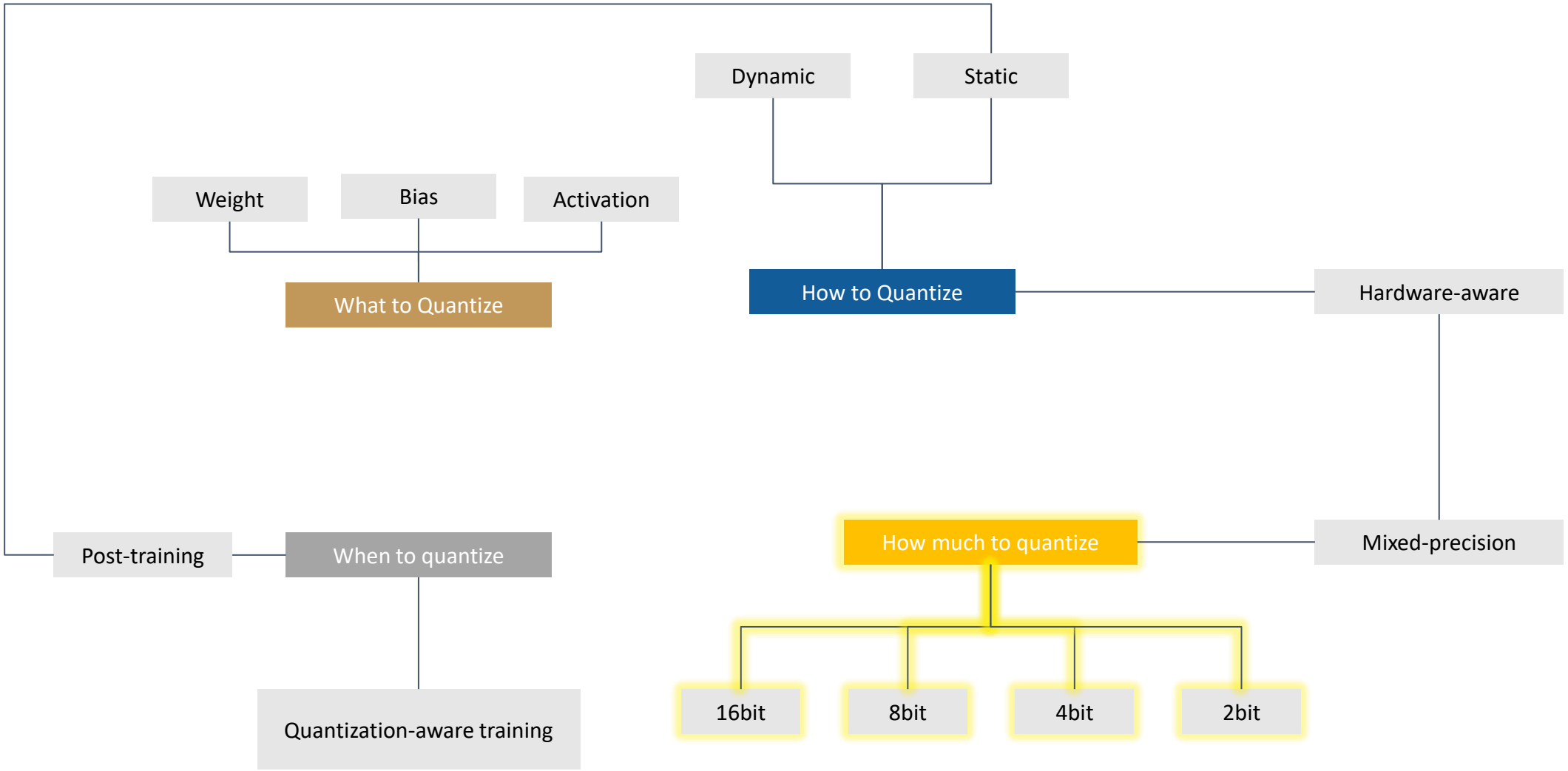
Quantization



Quantization



Quantization



How much to Quantize?

Quantization Types

Reduced Float

Hybrid Quantization

Integer Quantization

How much to Quantize?

Reduced float

- float16 parameters
- float16 computations

Reduced Float

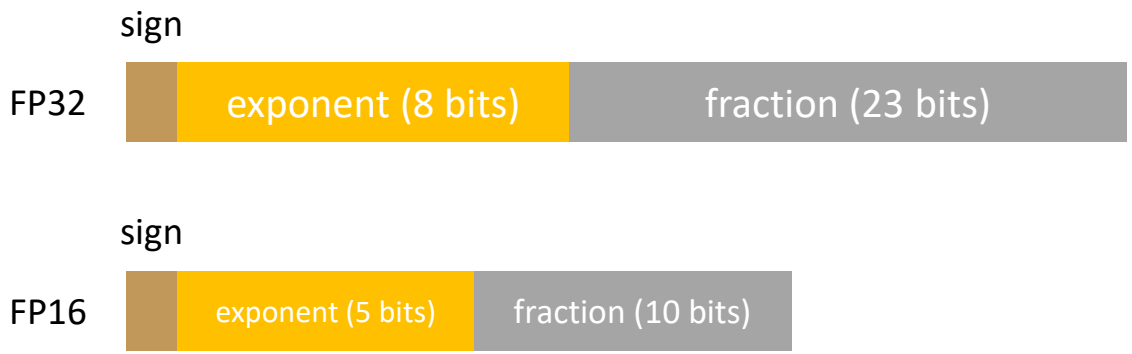
Hybrid Quantization

Integer Quantization

How much to Quantize?

Reduced float

- float16 parameters
- float16 computations



Reduced Float

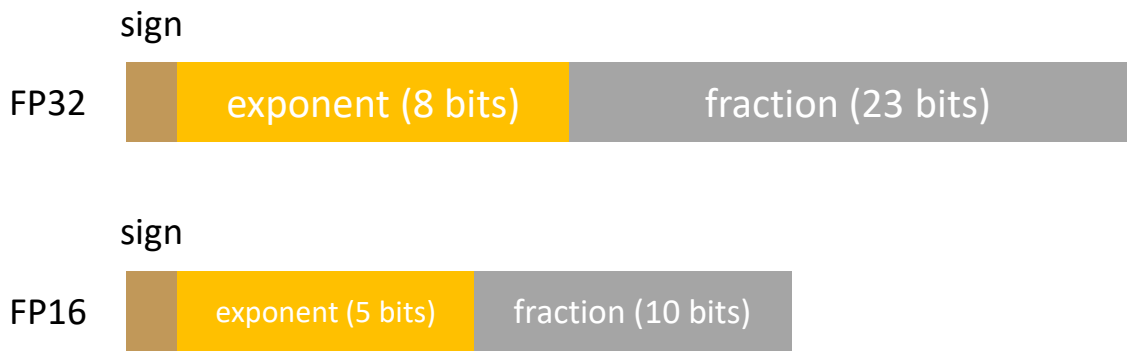
Hybrid Quantization

Integer Quantization

How much to Quantize?

Reduced float

- float16 parameters
- float16 computations



Benefits

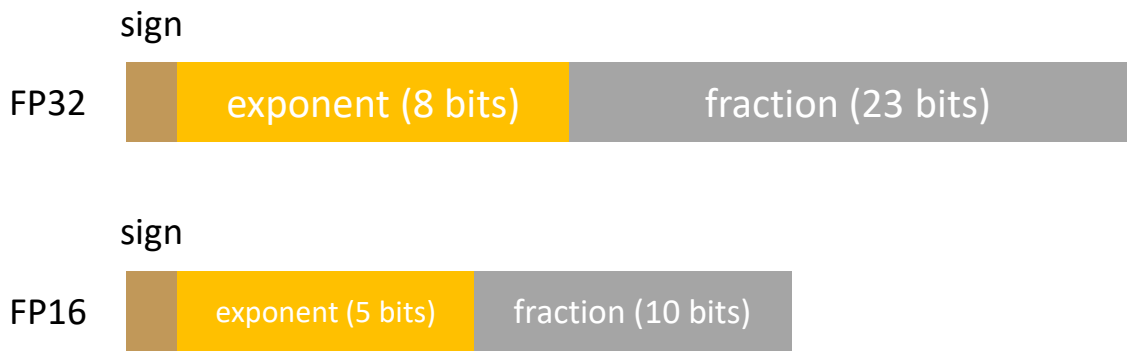
2x reduction in model parameters

(32 bit → 16 bit)

How much to Quantize?

Reduced float

- float16 parameters
- float16 computations



Benefits

2x reduction in model parameters (32 bit → 16 bit)

Potential future speed-ups faster as hardware enables optimized operations

Negligible accuracy loss

How much to Quantize?

Quantization Types

Reduced Float

Hybrid Quantization

Integer Quantization

How much to Quantize?

Hybrid quantization

- 8-bit integer weights
- 32-bit float biases & activations
- Integer and floating point computations

Reduced Float

Hybrid Quantization

Integer Quantization

How much to Quantize?

Hybrid quantization

- 8-bit integer weights
- 32-bit float biases & activations
- Integer and floating point computations

Benefits

4x reduction in model parameters (32 bit → 8 bit)

How much to Quantize?

Hybrid quantization

- 8-bit integer weights
- 32-bit float biases & activations
- Integer and floating point computations

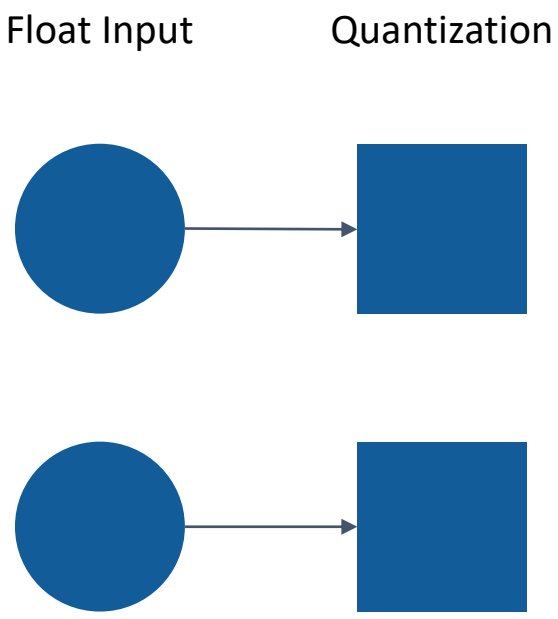
Benefits

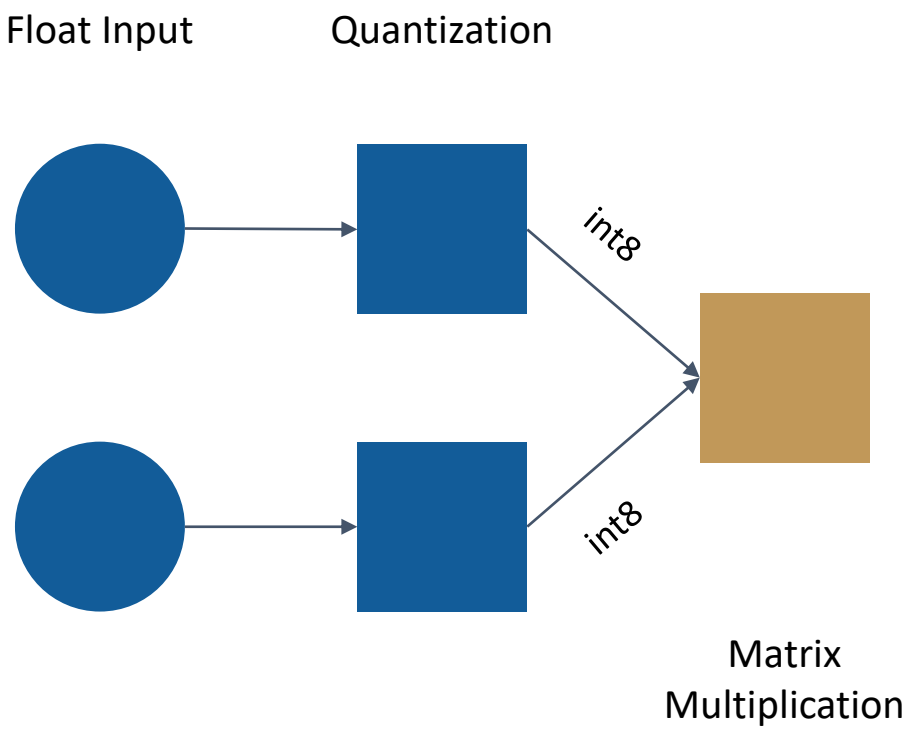
4x reduction in model parameters (32 bit → 8 bit)

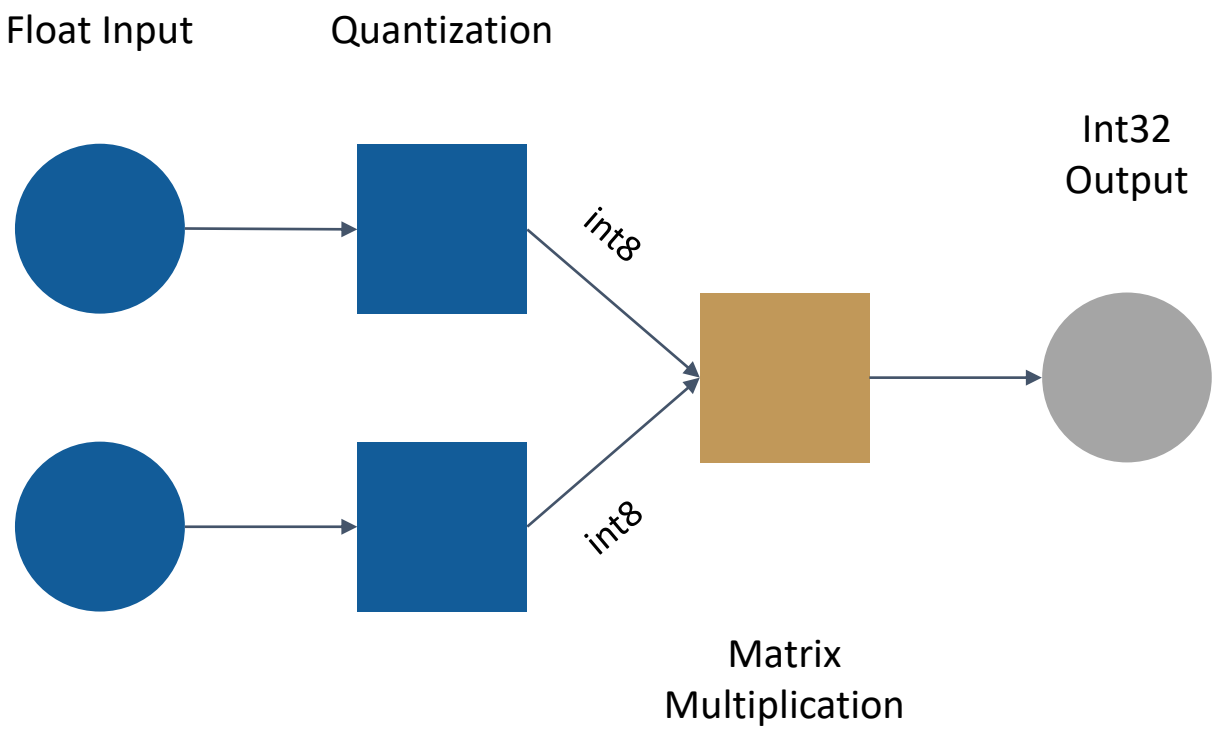
10-50% faster execution for Convolution Models (CPU hybrid v. CPU float)

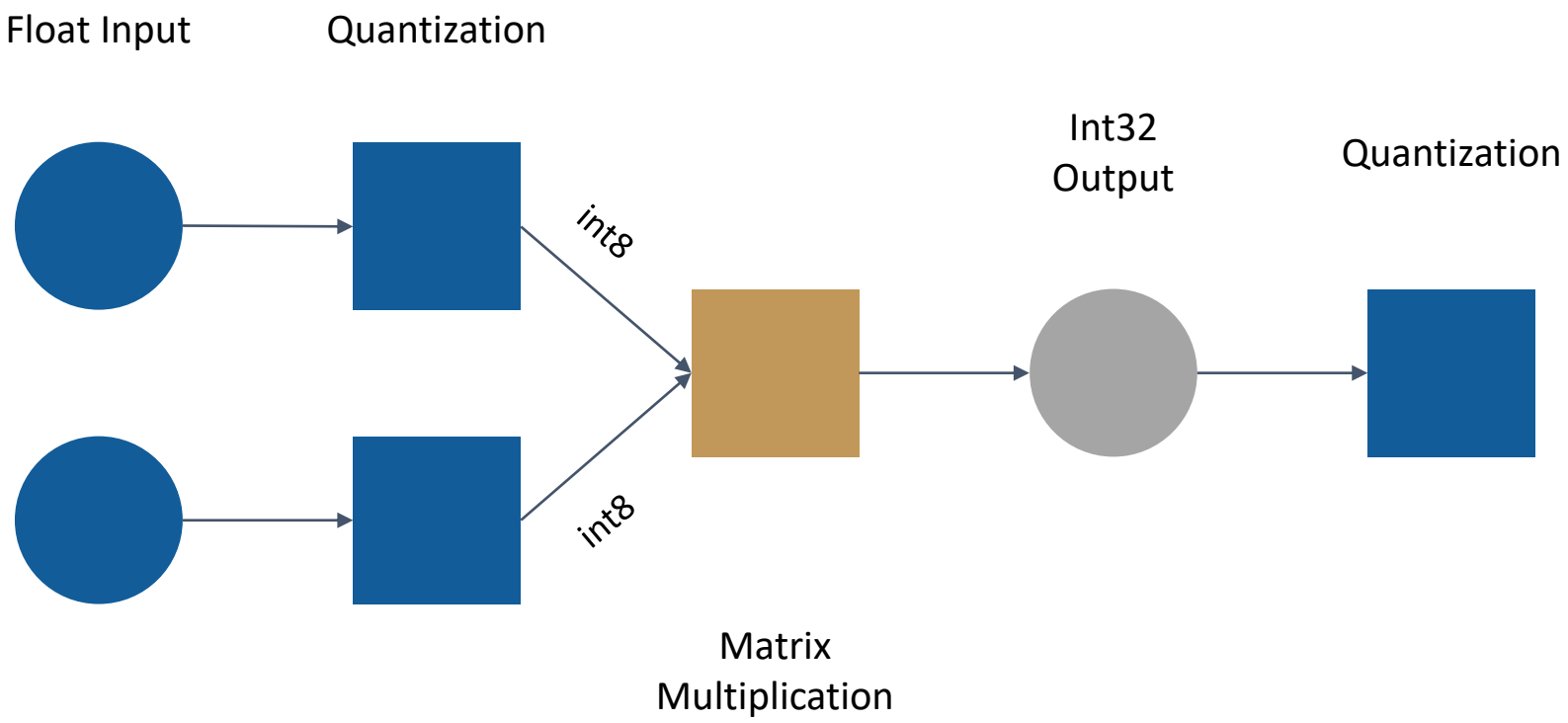
2-3x faster on fully-connected & RNN-based models (CPU quant v. CPU float)

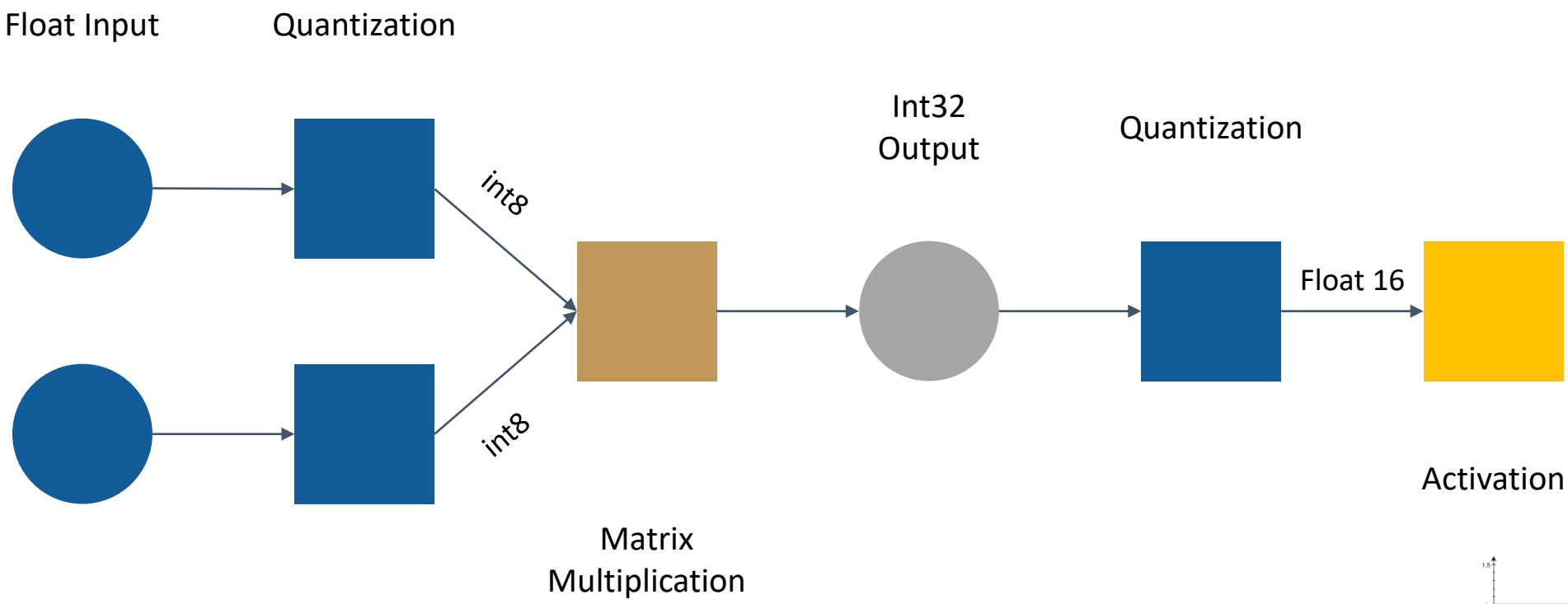
Float Input


Float Input
Quantization

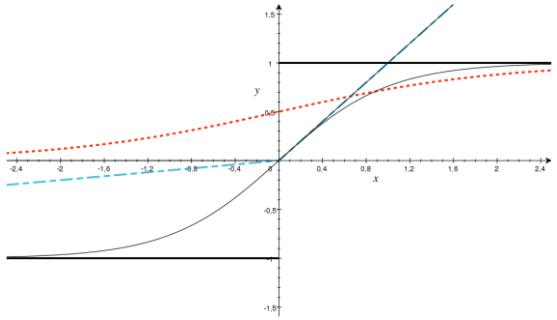

Float Input
Quantization
int8
int8
Matrix
Multiplication

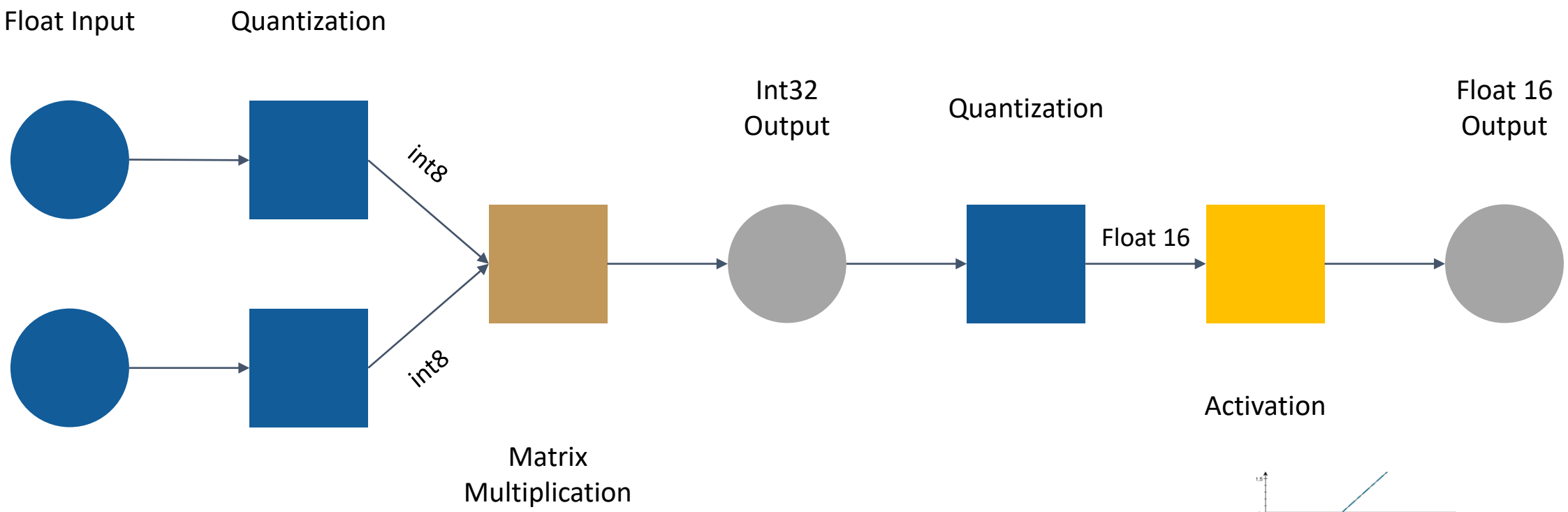

Float Input
Quantization
int8
int8
Int32
Output
Matrix
Multiplication

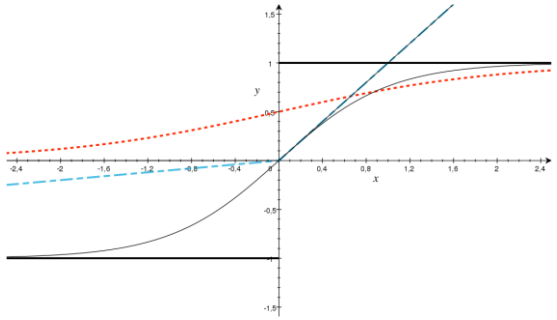

Float Input
Quantization
int8
int8
Matrix
Multiplication
Int32
Output
Quantization


Float Input
Quantization
int8
int8
Matrix
Multiplication
Int32
Output
Quantization
Float 16

Activation




Float Input
Quantization
int8
int8
Matrix
Multiplication
Int32
Output
Quantization
Float 16
Float 16
Output
Activation



How much to Quantize?

Hybrid quantization

- 8-bit integer weights
- 32-bit float biases & activations
- Integer and floating point computations

Benefits

4x reduction in model parameters (32 bit → 8 bit)

10-50% faster execution for Convolution Models (CPU hybrid v. CPU float)

2-3x faster on fully-connected & RNN-based models (CPU quant v. CPU float)

How much to Quantize?

Quantization Types

Reduced Float

Hybrid Quantization

Integer Quantization

How much to Quantize?

Integer quantization

- 8-bit integer weights
- 8 and 16-bit biases & activations
- Only integer computations

Reduced Float

Hybrid Quantization

Integer Quantization

How much to Quantize?

Integer quantization

- 8-bit integer weights
- 8 and 16-bit biases & activations
- Only integer computations

Benefits

4x reduction in model parameters (32 bit → 8 bit)

1.5x faster execution for Convolution Models (CPU integer v. CPU float)

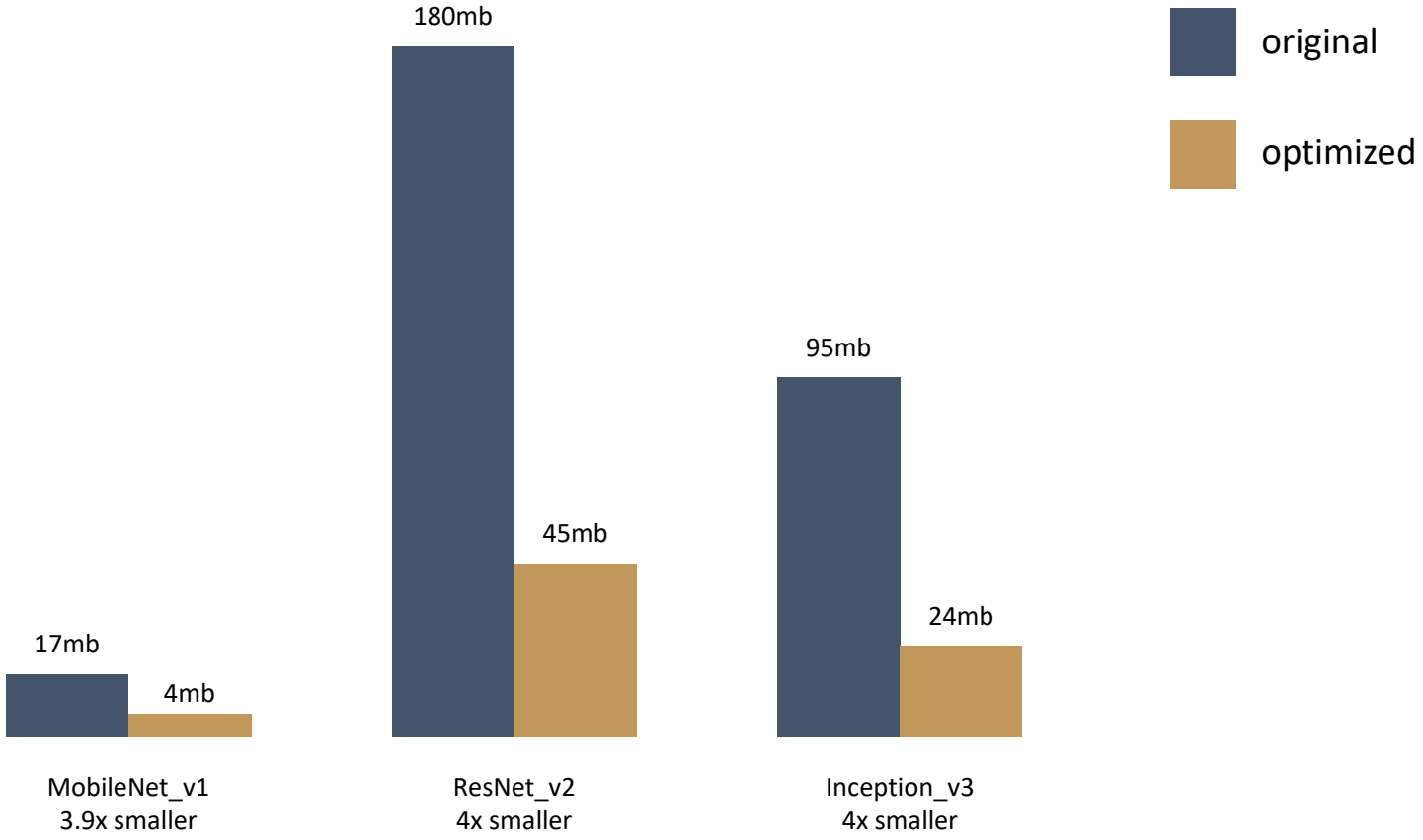
2-4x faster on fully-connected & RNN-based models (CPU quant v. CPU float)

Enables execution on ML accelerators (e.g., Edge-TPU)

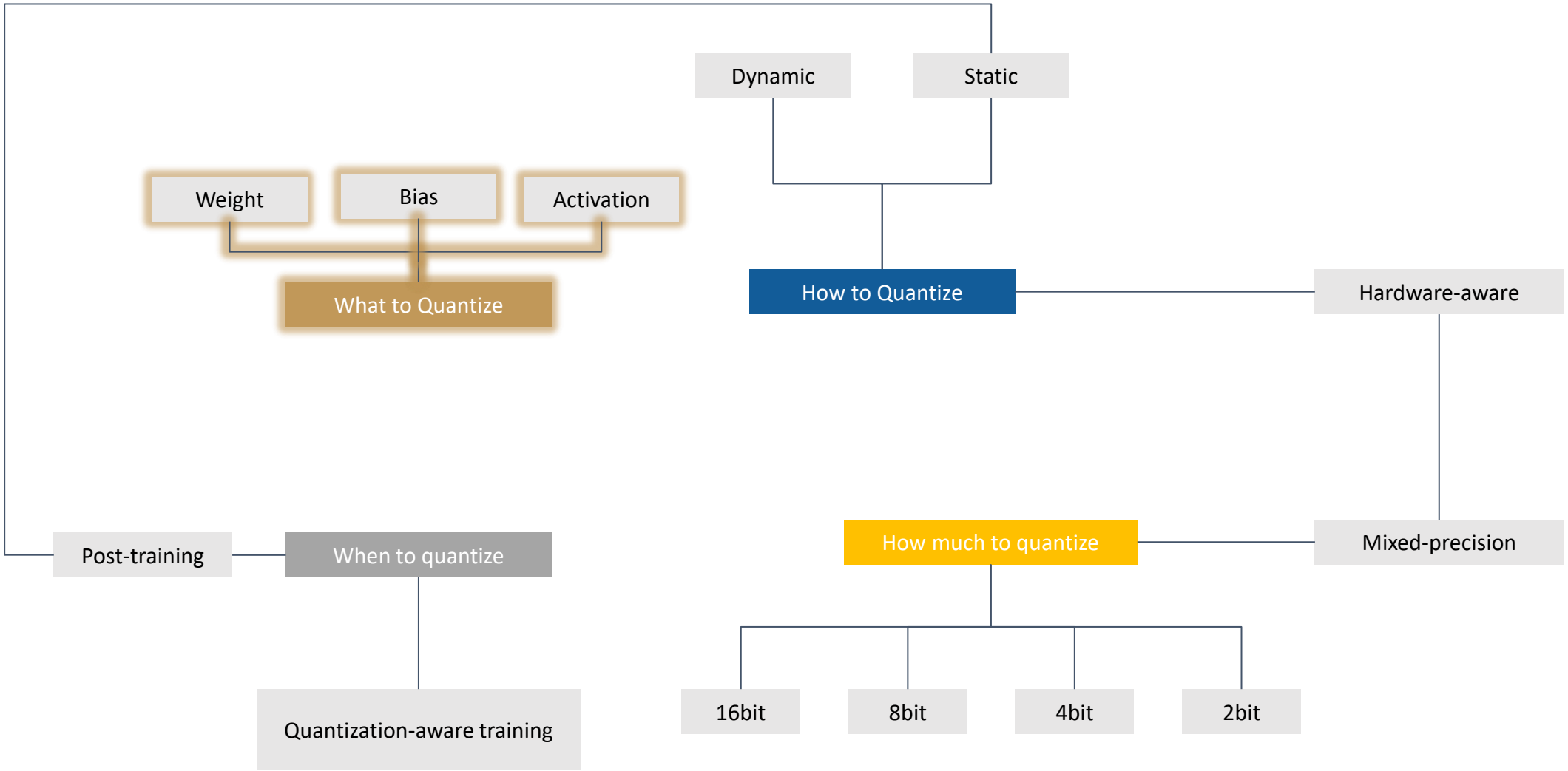
Comparison

Technique	Ease of use	Accuracy	Latency	Compatibility
Reduced float (post-training)	No data required	Negligible loss	Same or faster than float32	Float16 support or fallback to float32
"Hybrid" quantization (post-training)	No data required	Small loss (≤ float16)	Faster than float	Needs float and integer support
Integer quantization (post-training0	Unlabeled data	Accuracy ≤ hybrid	Fastest	Integer only
Integer quantization (during training)	Labeled training data	Accuracy ≥ integer	Fastest	Integer only

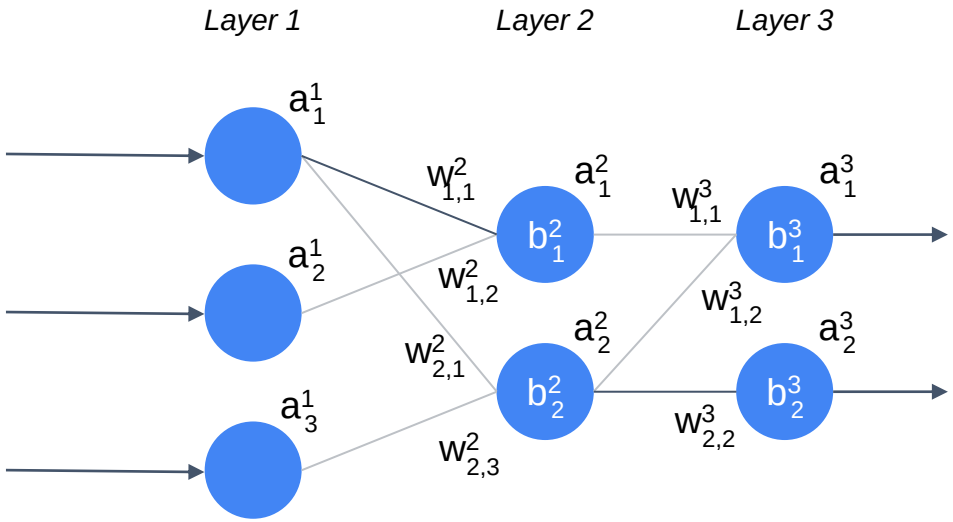
Model Size


180mb
95mb
45mb
24mb
17mb
4mb
original
optimized
MobileNet_v1
3.9x smaller
ResNet_v2
4x smaller
Inception_v3
4x smaller

Quantization

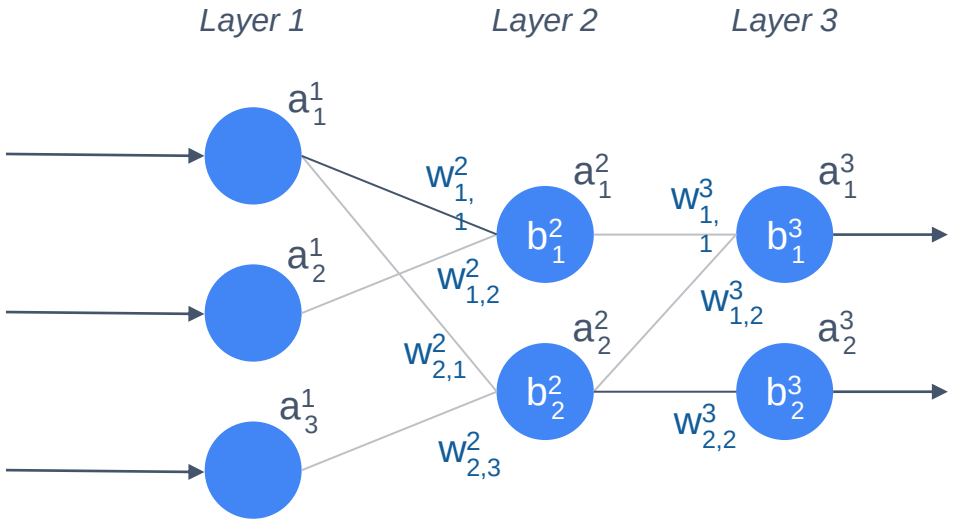


What to Quantize?



What to Quantize?

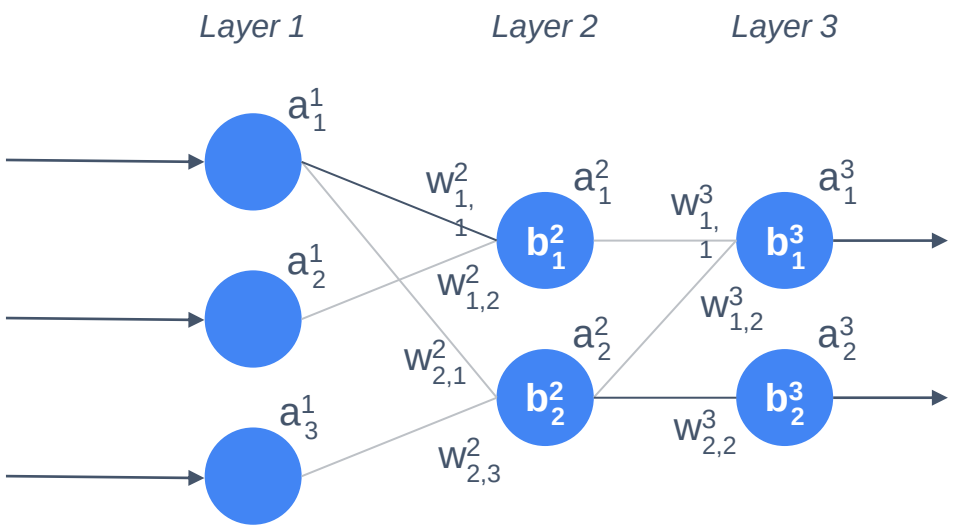
Weights



What to Quantize?

Weights

Biases

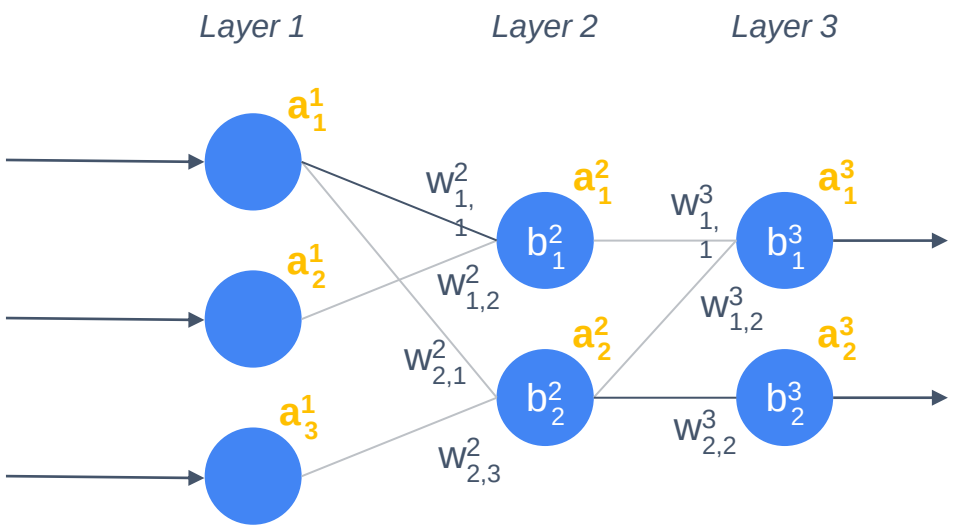


What to Quantize?

Weights

Biases

Activations

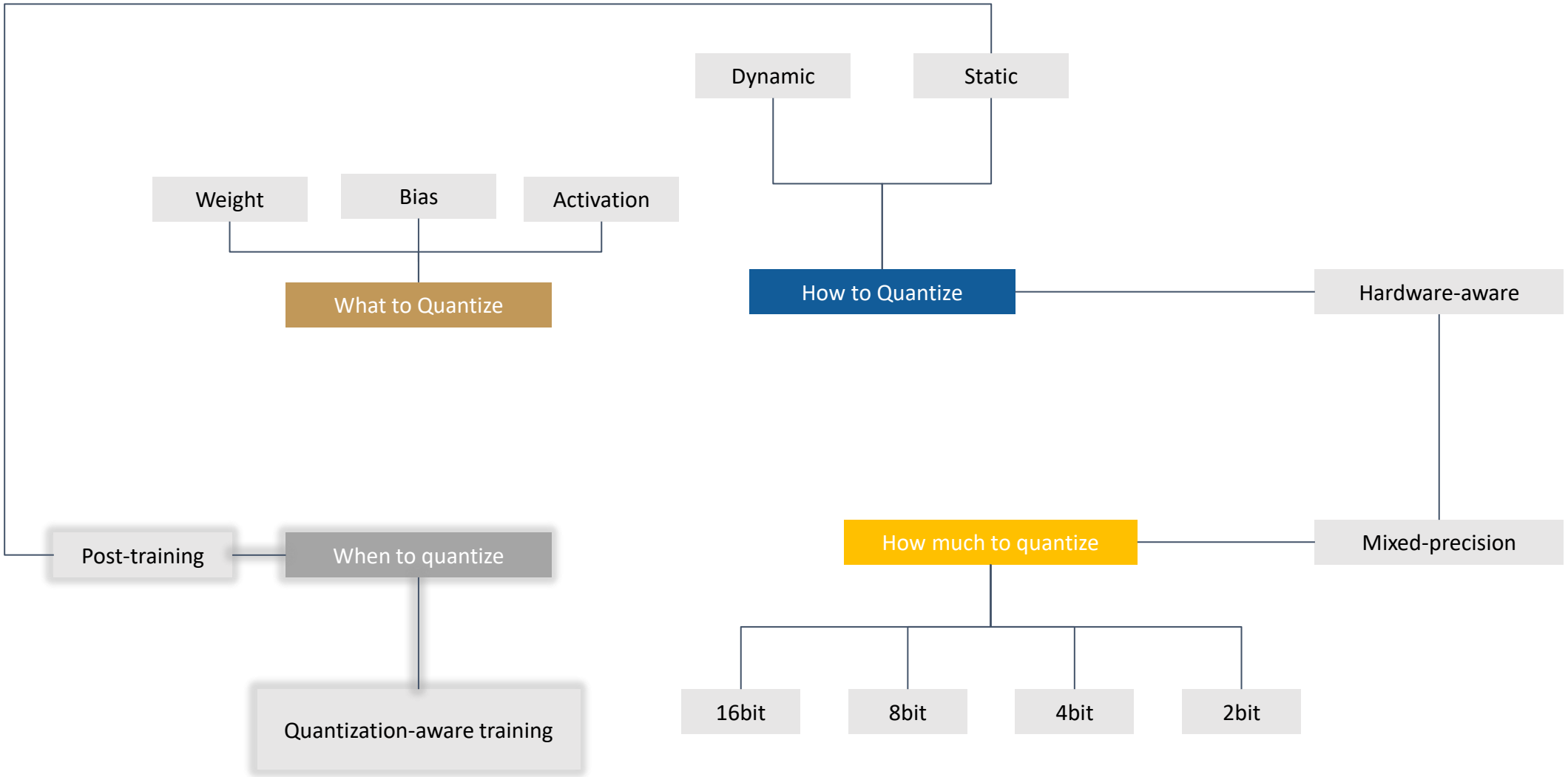


About Quantizing **Other NN Parts**?

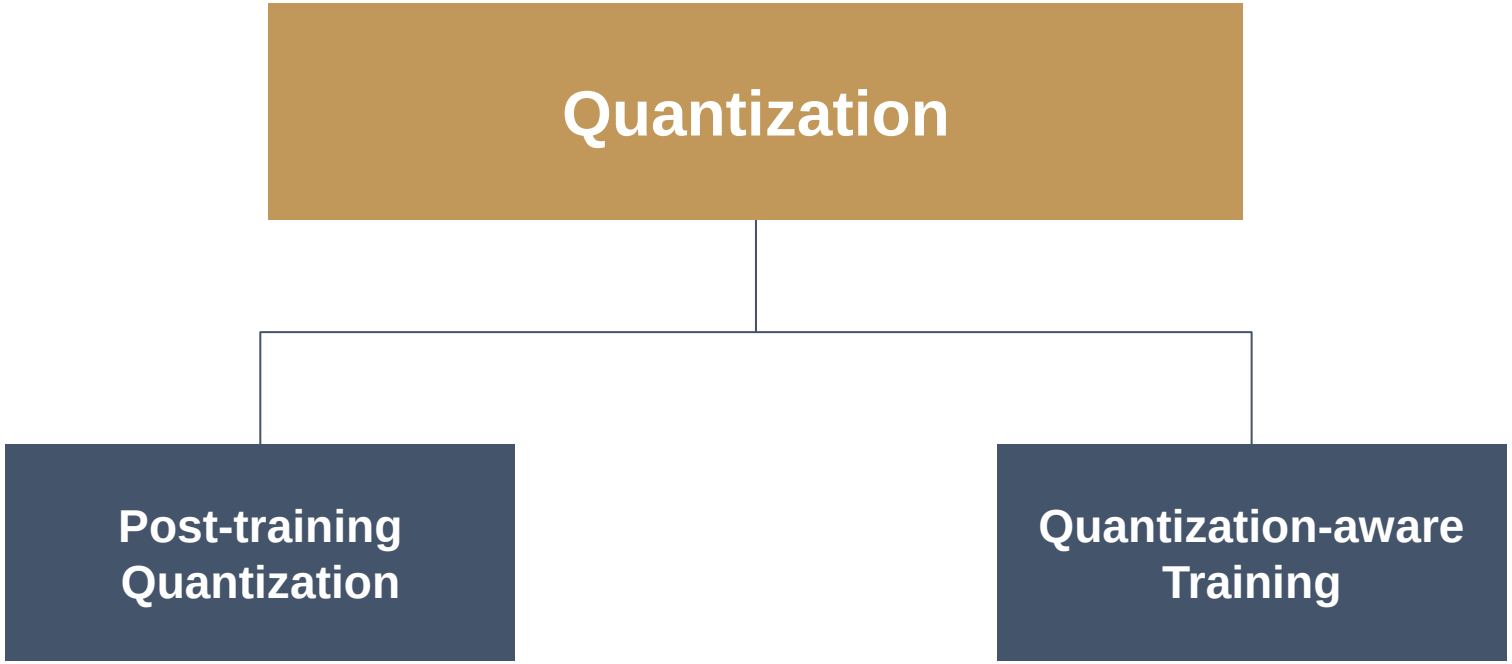
- Weights
- Activations
- Channels
- Tensors
- Layers
- ...

Every network has **something unique** for it, so the degree to which you can quantize (e.g., weights, activations) ***will vary***.

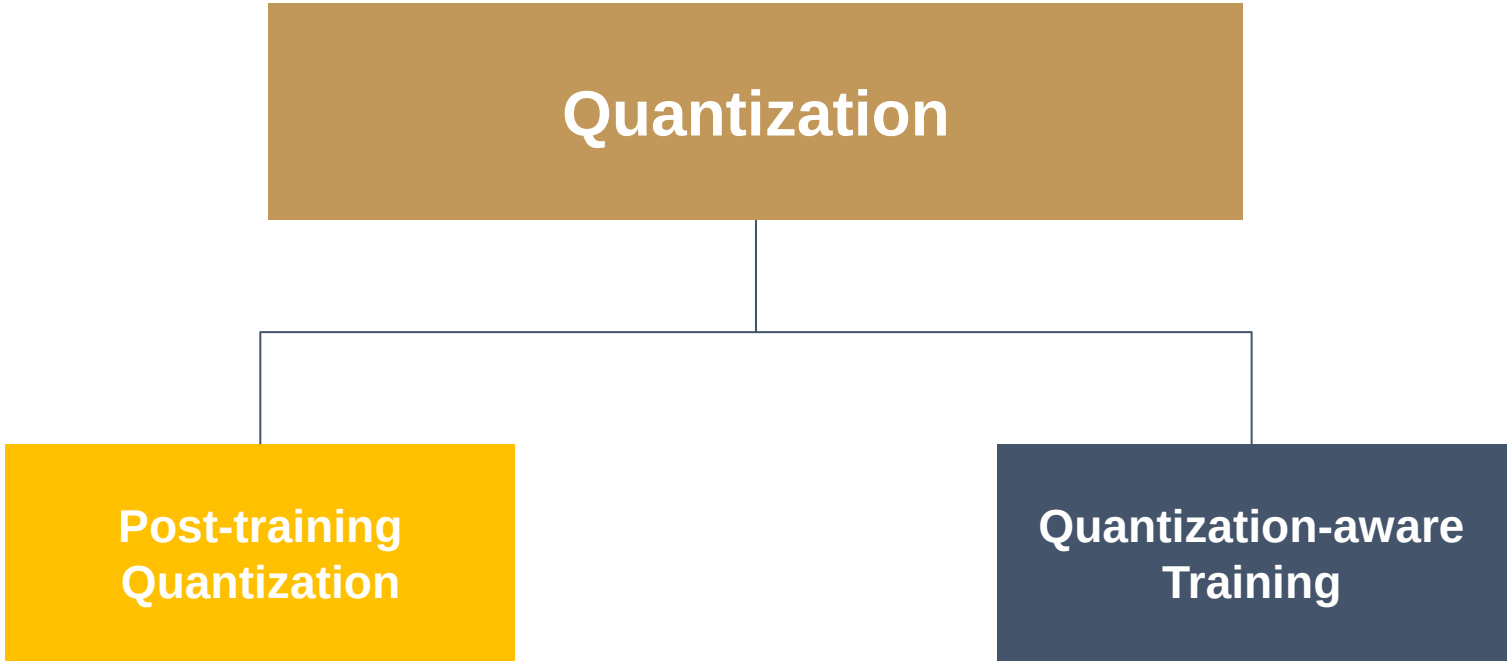
Quantization

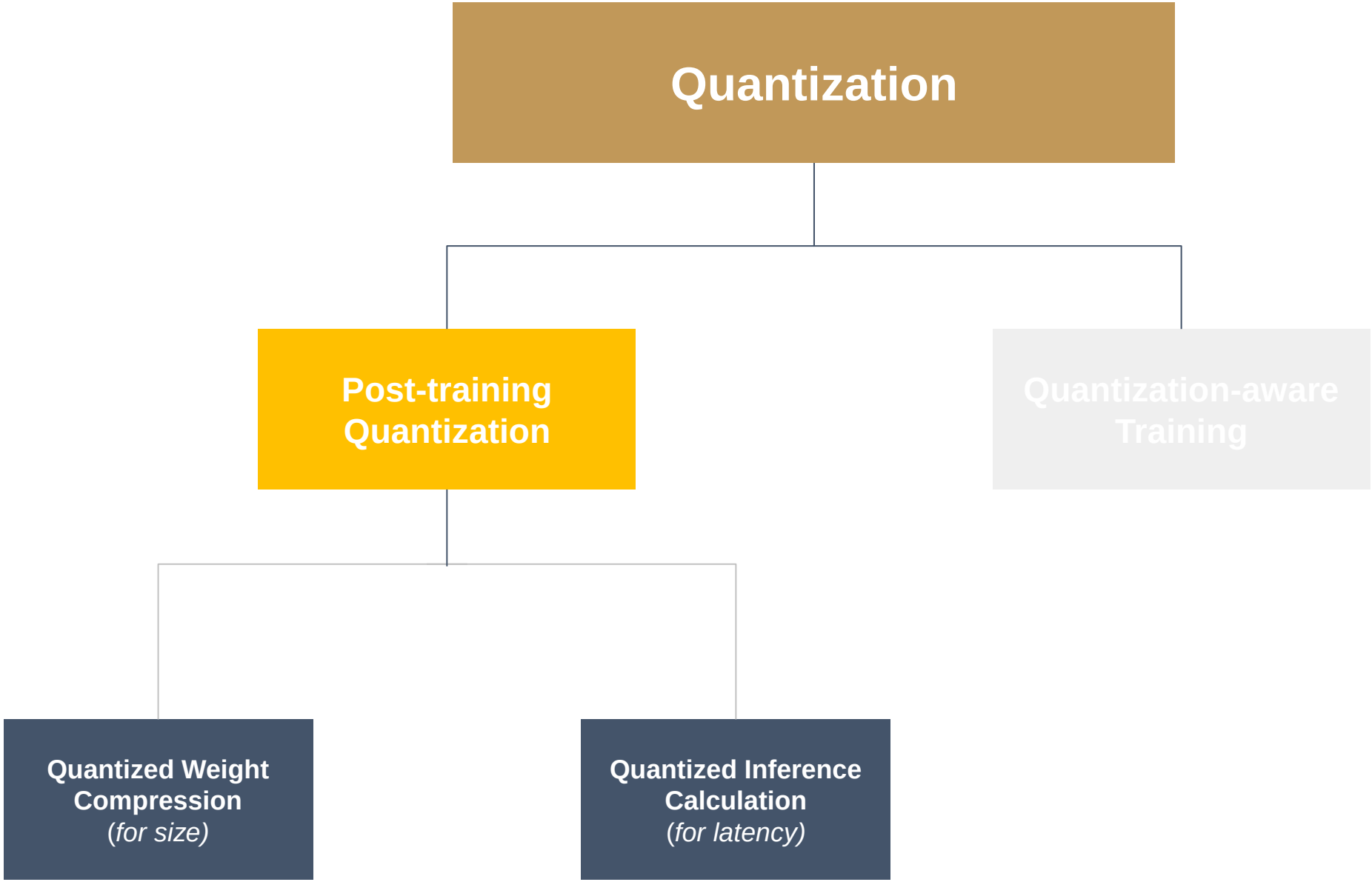


When to Quantize?



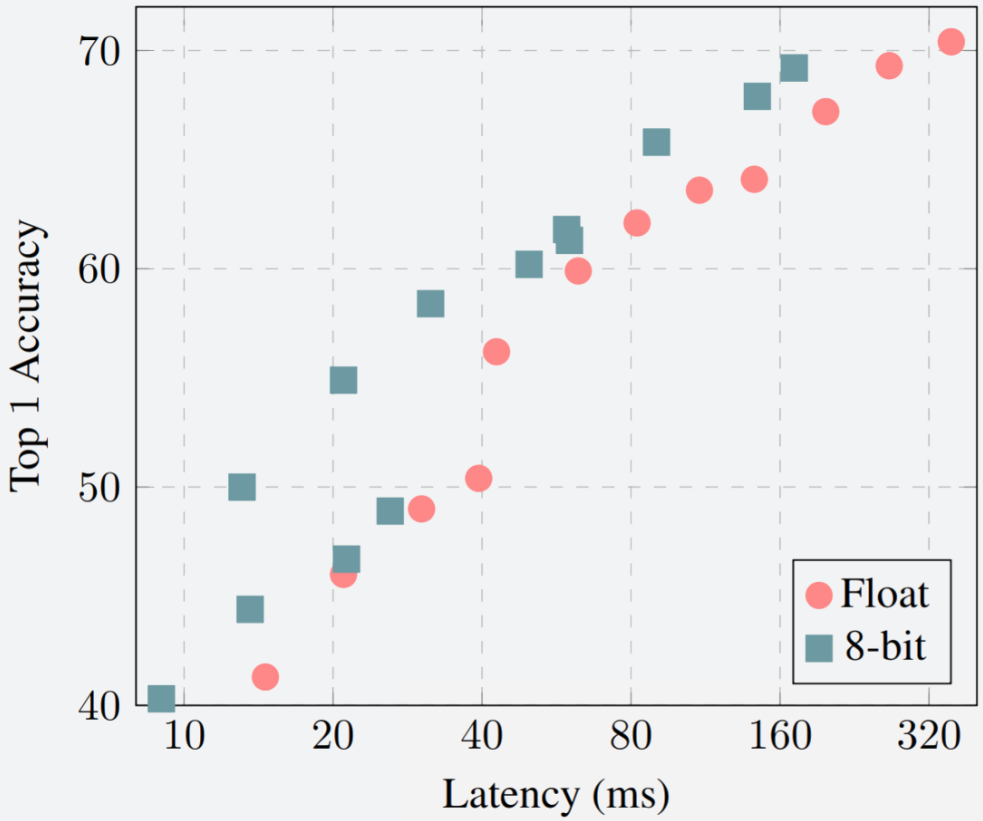
When to Quantize?




Quantization
Post-training Quantization
Quantization-aware Training
Quantized Weight Compression
(for size)
Quantized Inference Calculation
(for latency)

Accuracy-Latency **Trade-off**

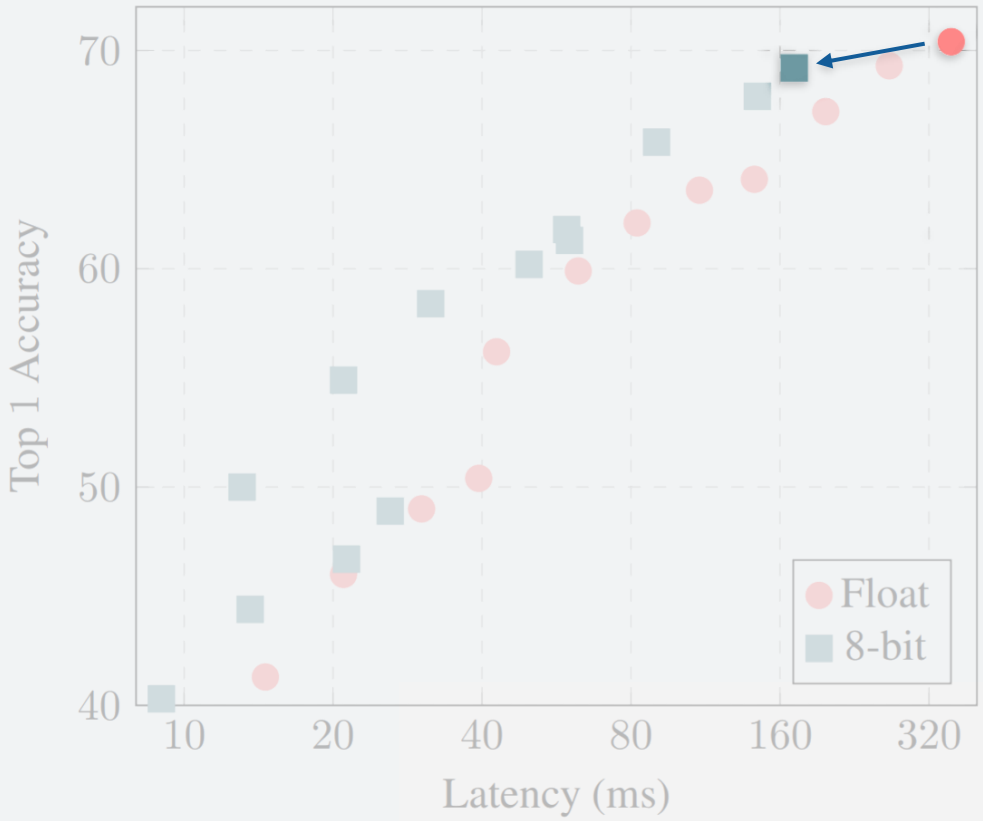
Quantization works well but performance but can suffer from **accuracy loss** during ***inference***.



•Source: “Accuracy-latency tradeoff with MobileNets” Benoit et al.

Accuracy-Latency **Trade-off**

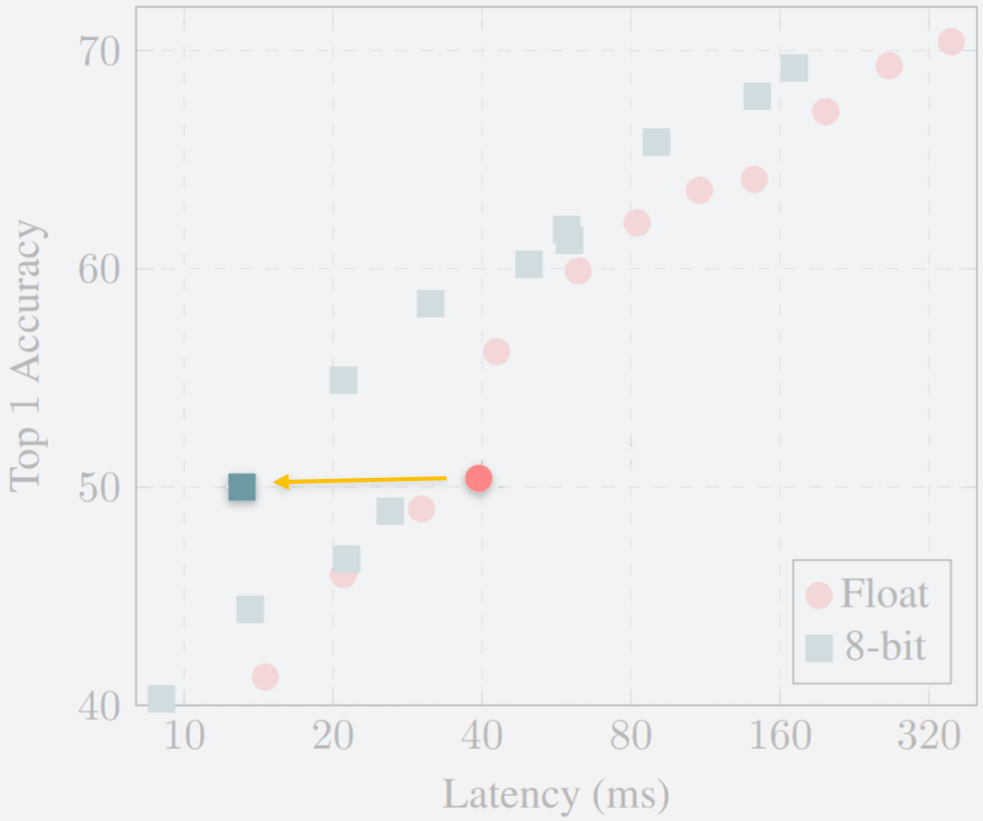
Quantization works well but performance but can suffer from **accuracy loss** during ***inference***.



•Source: “Accuracy-latency tradeoff with MobileNets” Benoit et al.

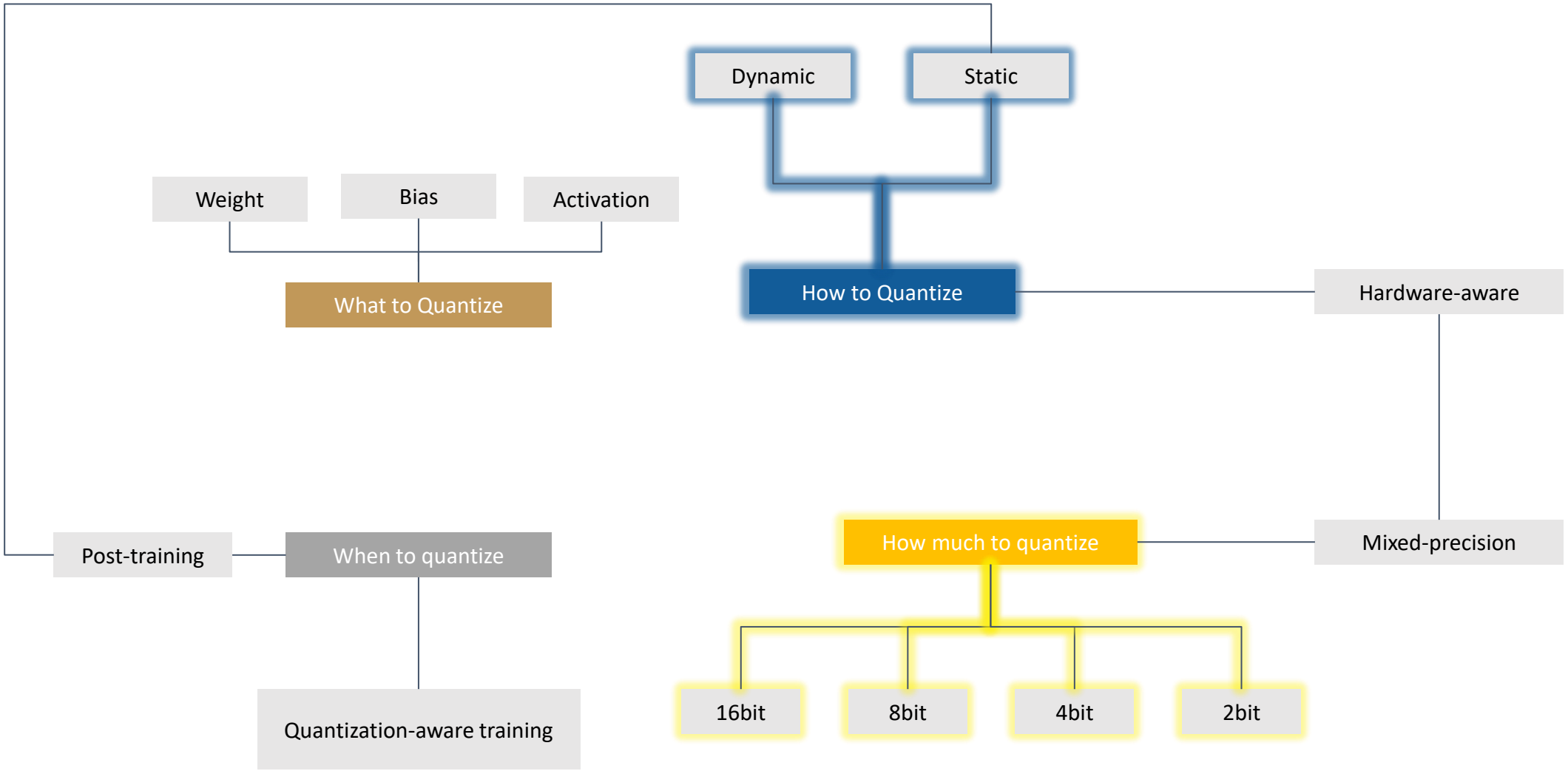
Accuracy-Latency **Trade-off**

Quantization works well but performance but can suffer from **accuracy loss** during ***inference***.



•Source: “Accuracy-latency tradeoff with MobileNets” Benoit et al.

Quantization



Quantization Types

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

3	0	2	1	3:	2.00
1	1	0	3	2:	1.50
0	3	1	0	1:	0.00
3	1	2	2	0:	-1.00

Clustering Quantization

1	-2	0	-1
-1	-1	-2	1
-2	1	-1	-2
1	-1	0	0

(− -1) × 1.07

Linear Quantization

Storage	Floating-Point Weights	Integer Weights; Floating-Point Codebook	Integer Weights
Computation	Floating-Point Arithmetic	Floating-Point Arithmetic	Integer Arithmetic

Clustering-based Weight Quantization

weight matrix

0.86	-1.88	-1.49	-0.51
-1.7	1.58	0.12	-2.38
1.9	-2.19	1.37	-2.79
1.92	0.46	2.85	0.61

Clustering-based Weight Quantization

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

Clustering-based Weight Quantization

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

~~2.09, 2.12, 1.92, 1.87~~

↓

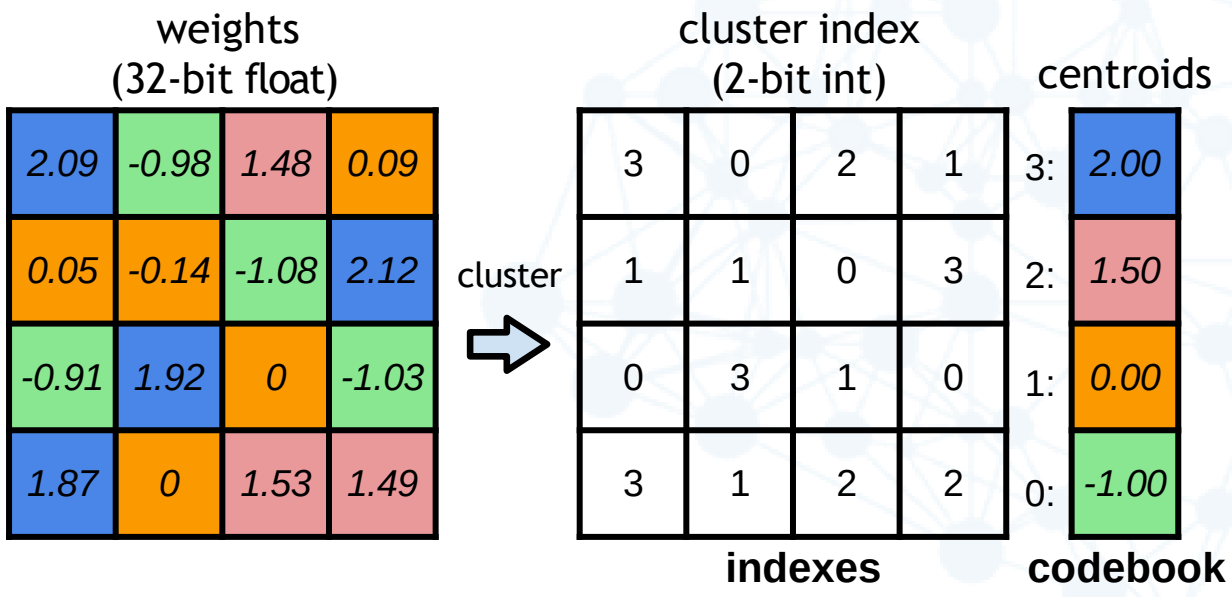
2.0

Clustering-based Weight Quantization

weights
(32-bit float)

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

Clustering-based Weight Quantization



Clustering-based Weight Quantization

weights
(32-bit float)

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

cluster

cluster index
(2-bit int)

3	0	2	1
1	1	0	3
0	3	1	0
3	1	2	2

indexes

centroids

3:	2.00
2:	1.50
1:	0.00
0:	-1.00

codebook

reconstructed weights
(32-bit float)

2.00	-1.00	1.50	0.00
0.00	0.00	-1.00	2.00
-1.00	2.00	0.00	-1.00
2.00	0.00	1.50	1.50

quantization error

0.09	0.02	-0.02	0.09
0.05	-0.14	-0.08	0.12
0.09	-0.08	0	-0.03
-0.13	0	0.03	-0.01

Clustering-based Weight Quantization

weights
(32-bit float)

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

cluster

cluster index
(2-bit int)

3	0	2	1
1	1	0	3
0	3	1	0
3	1	2	2

indexes

centroids

3:	2.00
2:	1.50
1:	0.00
0:	-1.00

codebook

reconstructed weights
(32-bit float)

2.00	-1.00	1.50	0.00
0.00	0.00	-1.00	2.00
-1.00	2.00	0.00	-1.00
2.00	0.00	1.50	1.50

quantization error

0.09	0.02	-0.02	0.09
0.05	-0.14	-0.08	0.12
0.09	-0.08	0	-0.03
-0.13	0	0.03	-0.01

storage

32 bit × 16
= 512 bit = 64 B

2 bit × 16
= 32 bit = 4 B

+

32 bit × 4
= 128 bit = 16 B

= 20 B

3.2 × smaller

Clustering-based Weight Quantization

weights
(32-bit float)

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

cluster →

cluster index
(2-bit int)

3	0	2	1
1	1	0	3
0	3	1	0
3	1	2	2

indexes

centroids

3:	2.00
2:	1.50
1:	0.00
0:	-1.00

codebook

reconstructed weights
(32-bit float)

2.00	-1.00	1.50	0.00
0.00	0.00	-1.00	2.00
-1.00	2.00	0.00	-1.00
2.00	0.00	1.50	1.50

quantization error

0.09	0.02	-0.02	0.09
0.05	-0.14	-0.08	0.12
0.09	-0.08	0	-0.03
-0.13	0	0.03	-0.01

storage

32 bit × 16
= 512 bit = 64 B

2 bit × 16
= 32 bit = 4 B

+

32 bit × 4
= 128 bit = 16 B

= 20 B

3.2 × smaller

Assume N-bit quantization, and #parameters = $M >> 2^N$.

32 bit × M
= $32M$ bit

N bit × M
= NM bit

~~32 bit × 2^N~~
~~= 2^{N+5} bit~~

$32/N$ × smaller

Clustering-based Weight Quantization

Fine-tuning Quantized Weights

weights
(32-bit float)

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

cluster

cluster index
(2-bit int)

3	0	2	1
1	1	0	3
0	3	1	0
3	1	2	2

centroids

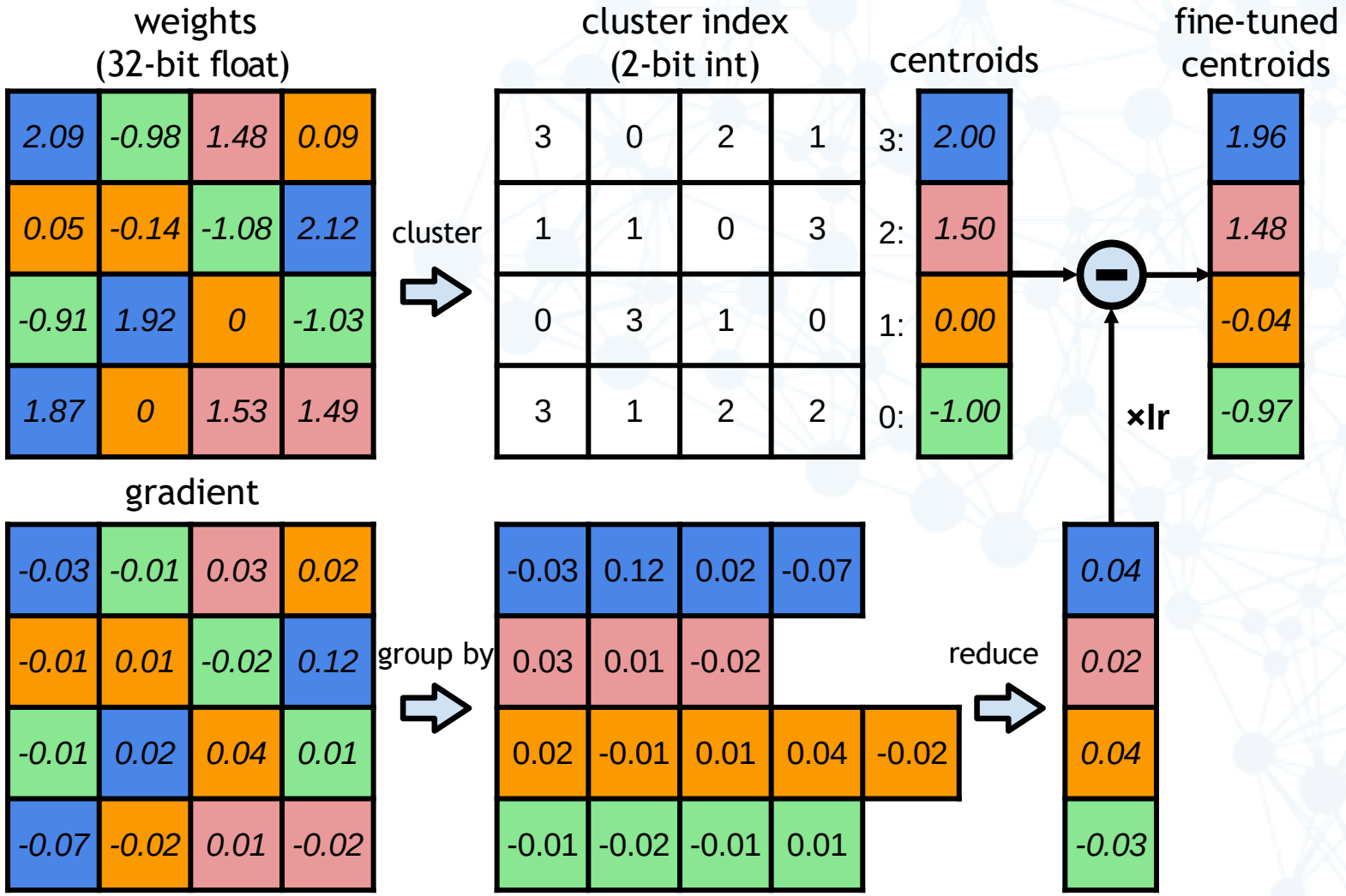
3:	2.00
2:	1.50
1:	0.00
0:	-1.00

gradient

-0.03	-0.01	0.03	0.02
-0.01	0.01	-0.02	0.12
-0.01	0.02	0.04	0.01
-0.07	-0.02	0.01	-0.02

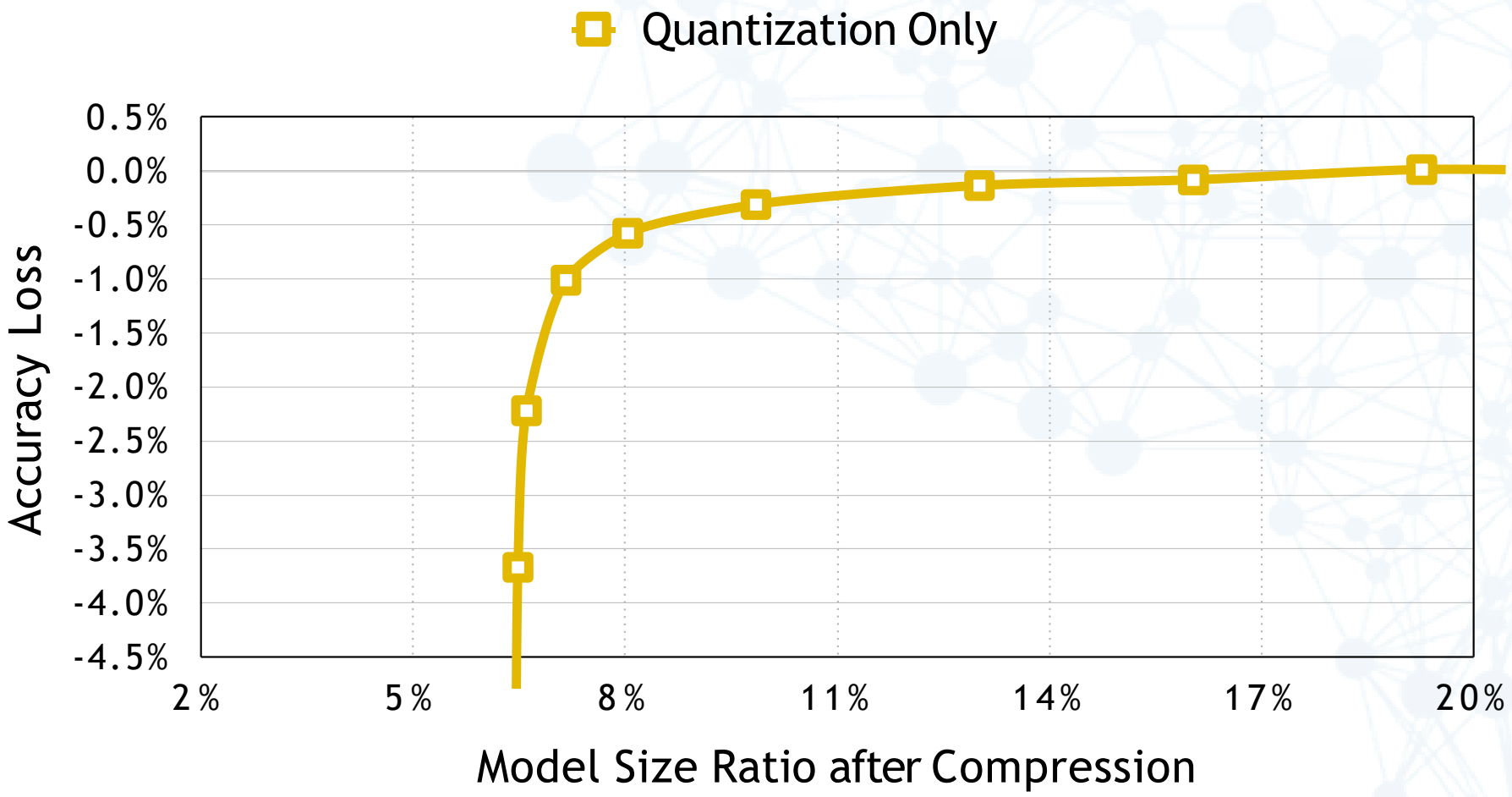
Clustering-based Weight Quantization

Fine-tuning Quantized Weights



K-Means-based Weight Quantization

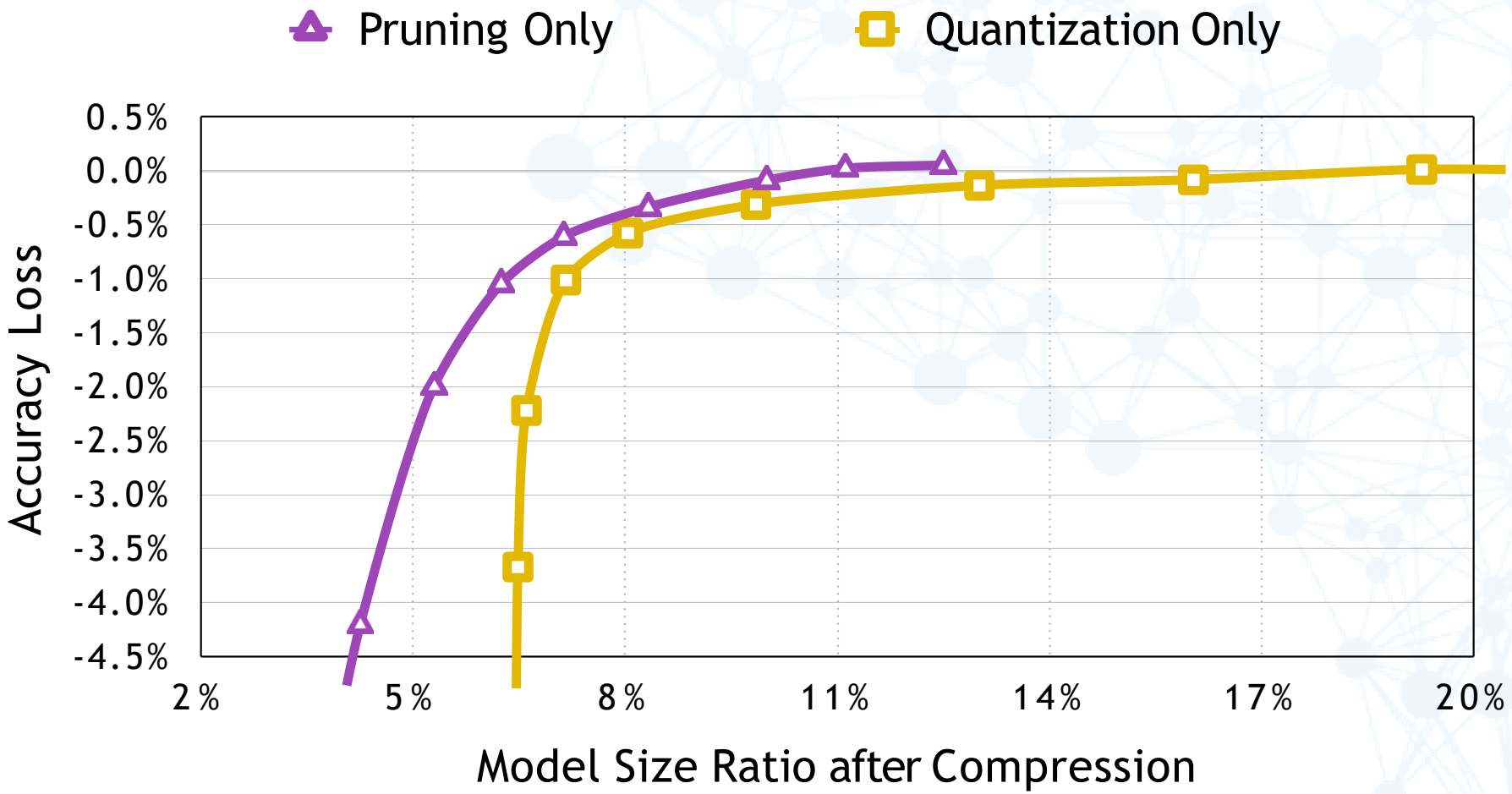
Accuracy vs. compression rate for AlexNet on ImageNet dataset



Deep Compression [Han *et al.*, ICLR 2016]

K-Means-based Weight Quantization

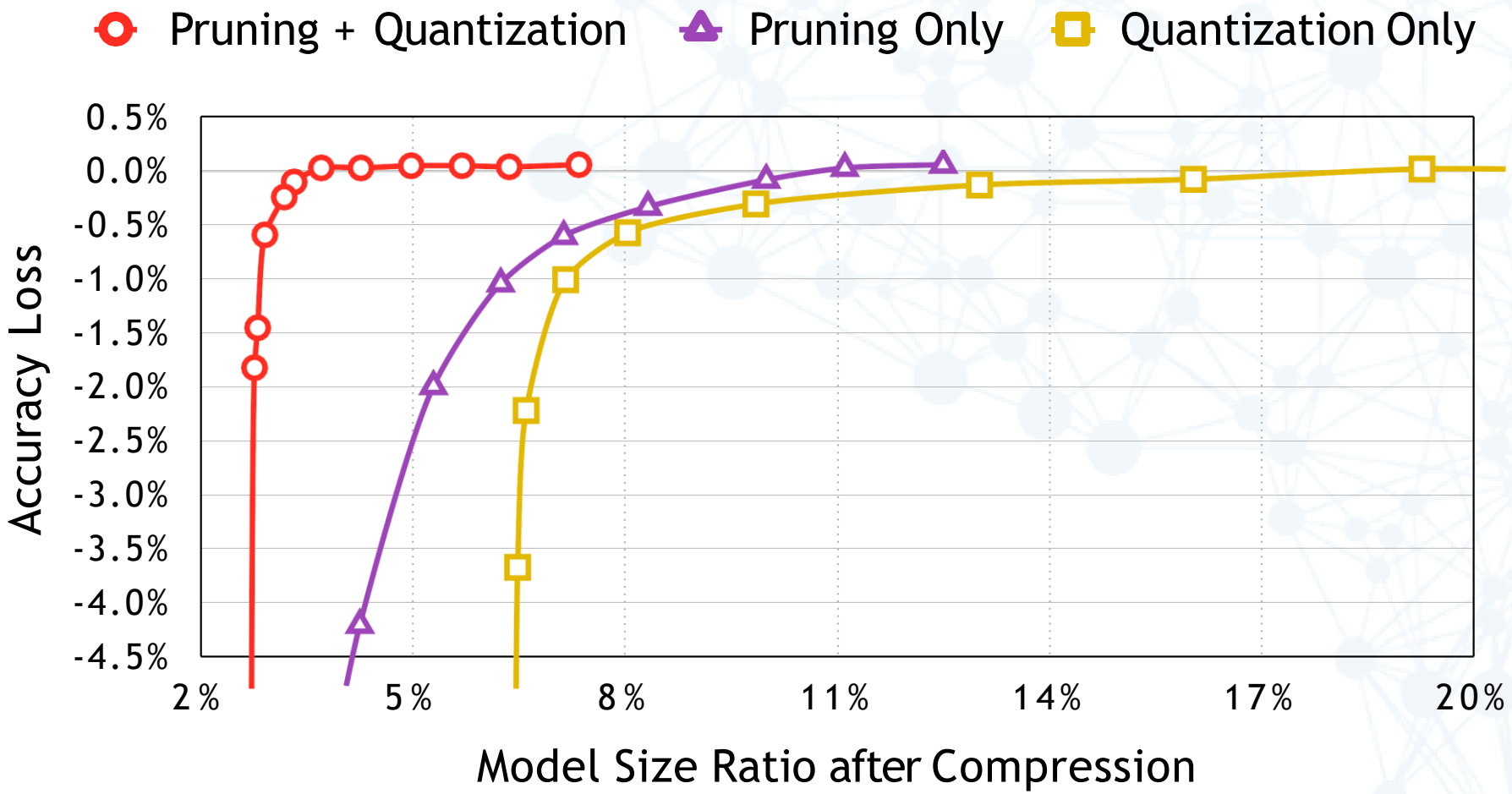
Accuracy vs. compression rate for AlexNet on ImageNet dataset



Deep Compression [Han *et al.*, ICLR 2016]

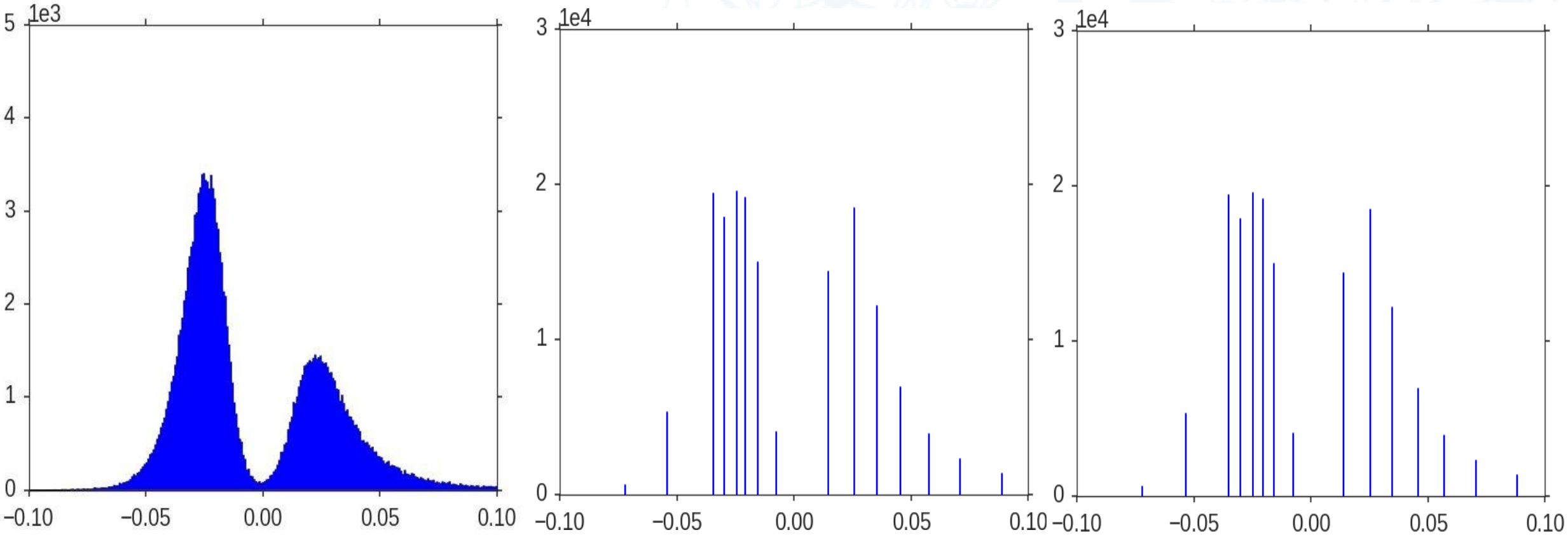
K-Means-based Weight Quantization

Accuracy vs. compression rate for AlexNet on ImageNet dataset

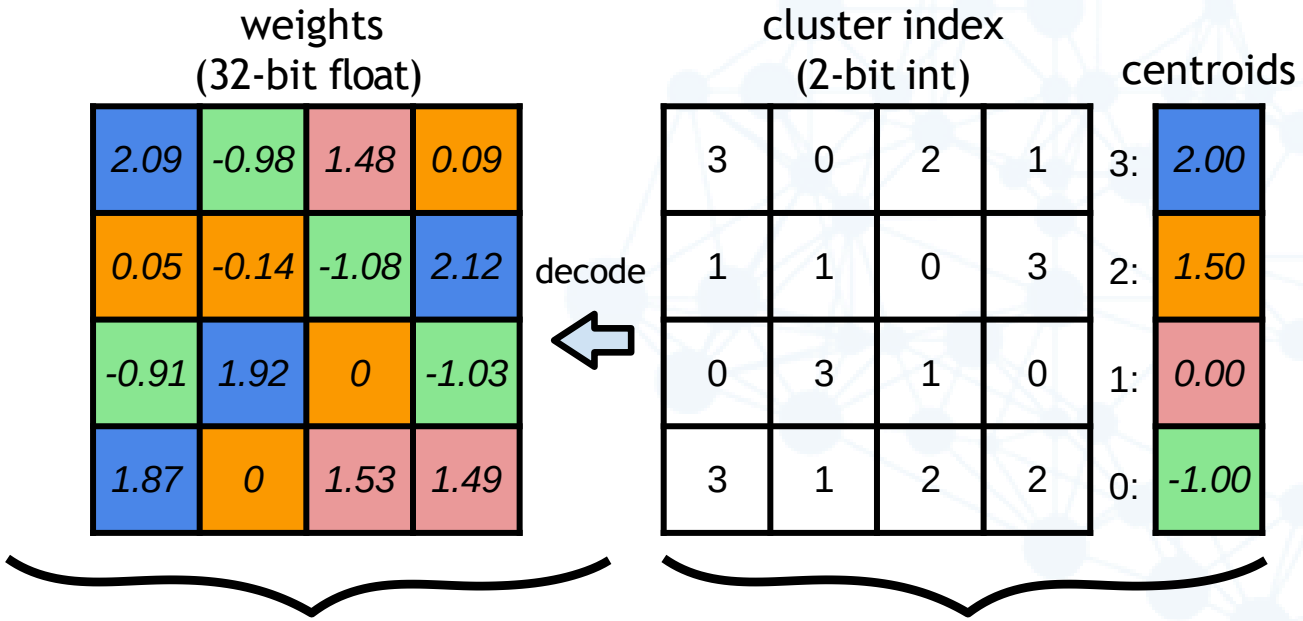


Deep Compression [Han *et al.*, ICLR 2016]

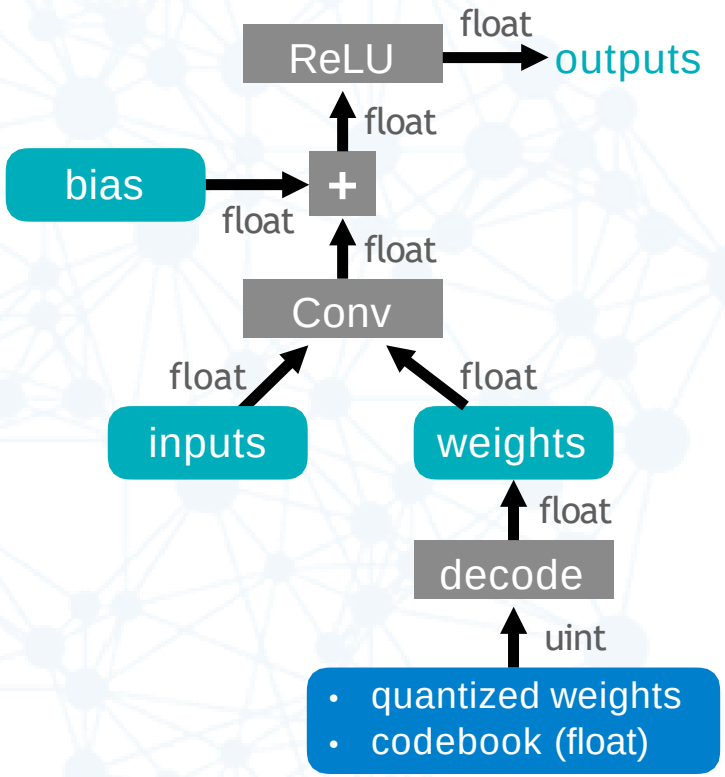
Weights: before, after, and fine-tuning



K-Means-based Weight Quantization



During Computation **In Storage**



- The weights are decompressed using a lookup table (*i.e.*, codebook) during runtime inference.
- K-Means-based Weight Quantization only saves storage cost of a neural network model.
  - All the computation and memory access are still floating-point.

Quantization Types

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

3	0	2	1	3:	2.00
1	1	0	3	2:	1.50
0	3	1	0	1:	0.00
3	1	2	2	0:	-1.00

Clustering Quantization

1	-2	0	-1
-1	-1	-2	1
-2	1	-1	-2
1	-1	0	0

(− -1) × 1.07

Linear Quantization

Storage	Floating-Point Weights	Integer Weights; Floating-Point Codebook	Integer Weights
Computation	Floating-Point Arithmetic	Floating-Point Arithmetic	Integer Arithmetic

What is Linear Quantization?

weights
(32-bit float)

2.09	*-0.98*	*1.48*	*0.09*
0.05	*-0.14*	*-1.08*	*2.12*
-0.91	*1.92*	*0*	*-1.03*
1.87	*0*	*1.53*	*1.49*

What is Linear Quantization?

An affine mapping of integers to real numbers

weights (32-bit float)

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

⇨ (quantized weights (2-bit signed int)

1	-2	0	-1
-1	-1	-2	1
-2	1	-1	-2
1	-1	0	0

− zero point (2-bit signed int) **-1**) × scale (32-bit float) **1.07** =

we will learn how to determine these parameters later

reconstructed weights (32-bit float)

2.14	-1.07	1.07	0
0	0	-1.07	2.14
-1.07	2.14	0	-1.07
2.14	0	1.07	1.07

quantization error

-0.05	0.09	0.41	0.09
0.05	-0.14	-0.01	-0.02
0.16	-0.22	0	0.04
-0.27	0	0.46	0.42

Binary	Decimal
01	1
00	0
11	-1
10	-2

Linear Quantization

An affine mapping of integers to real numbers $r = S(q - Z)$

weights (32-bit float)			
2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

⇨ (quantized weights (2-bit signed int) − zero point (2-bit signed int)) × scale (32-bit float)

1	-2	0	-1
-1	-1	-2	1
-2	1	-1	-2
1	-1	0	0

− -1) × 1.07

$$r = (q - Z) \times S$$

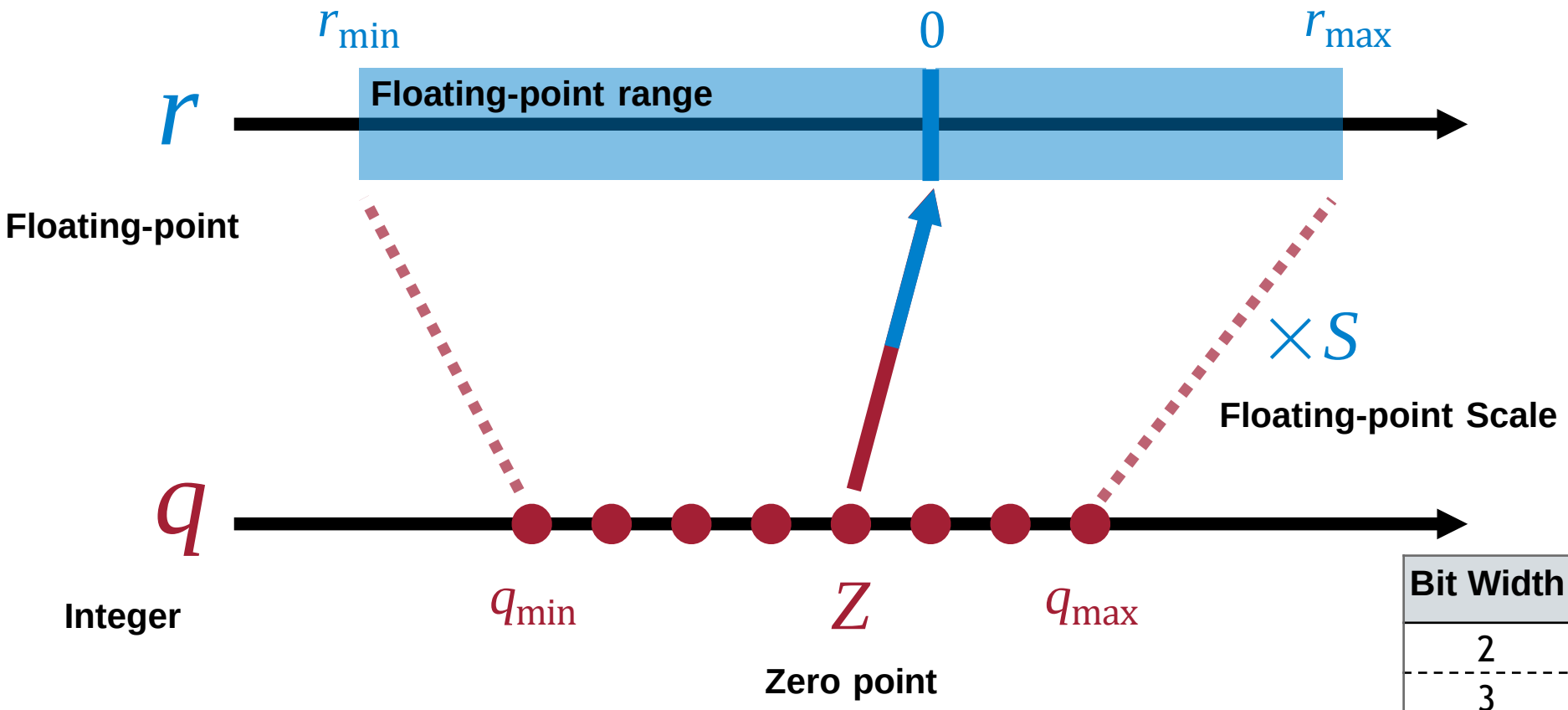
Floating-point **Integer** **Integer** **Floating-point**

- quantization parameter
- allow real number *r=0* be exactly representable by a quantized integer

- quantization parameter

Linear Quantization

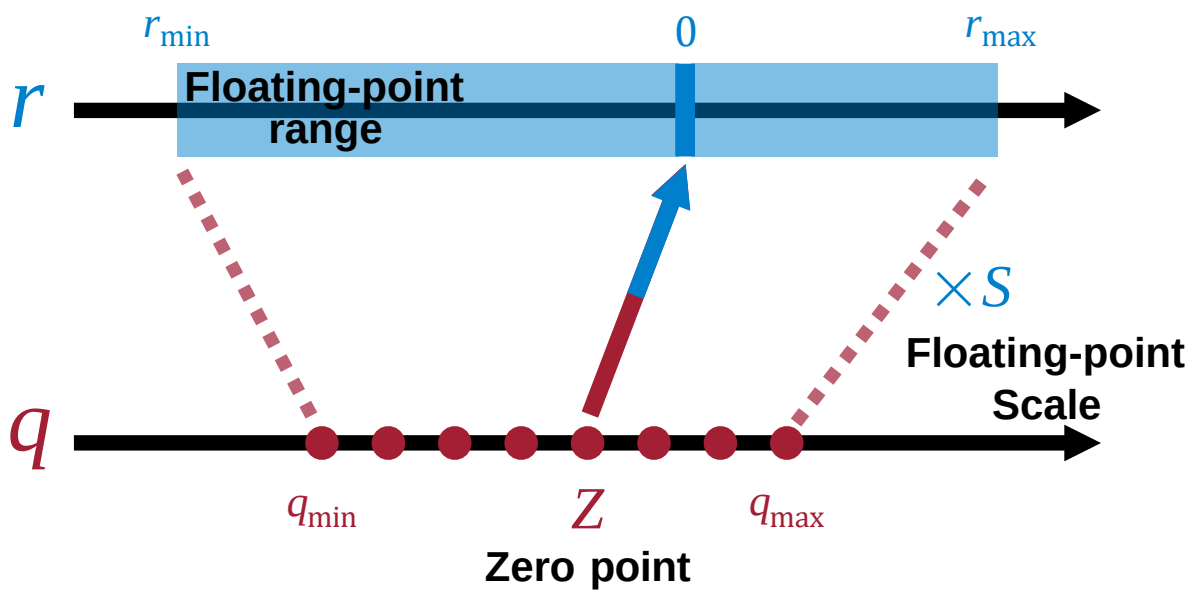
An affine mapping of integers to real numbers $r = S(q - Z)$



Bit Width	q_{min}	q_{max}
2	-2	1
3	-4	3
4	-8	7
N	-2^{N-1}	$2^{N-1}-1$

Scale of Linear Quantization

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$



$$r_{\max} = S\left(q_{\max} - Z\right)$$

$$r_{\min} = S\left(q_{\min} - Z\right)$$

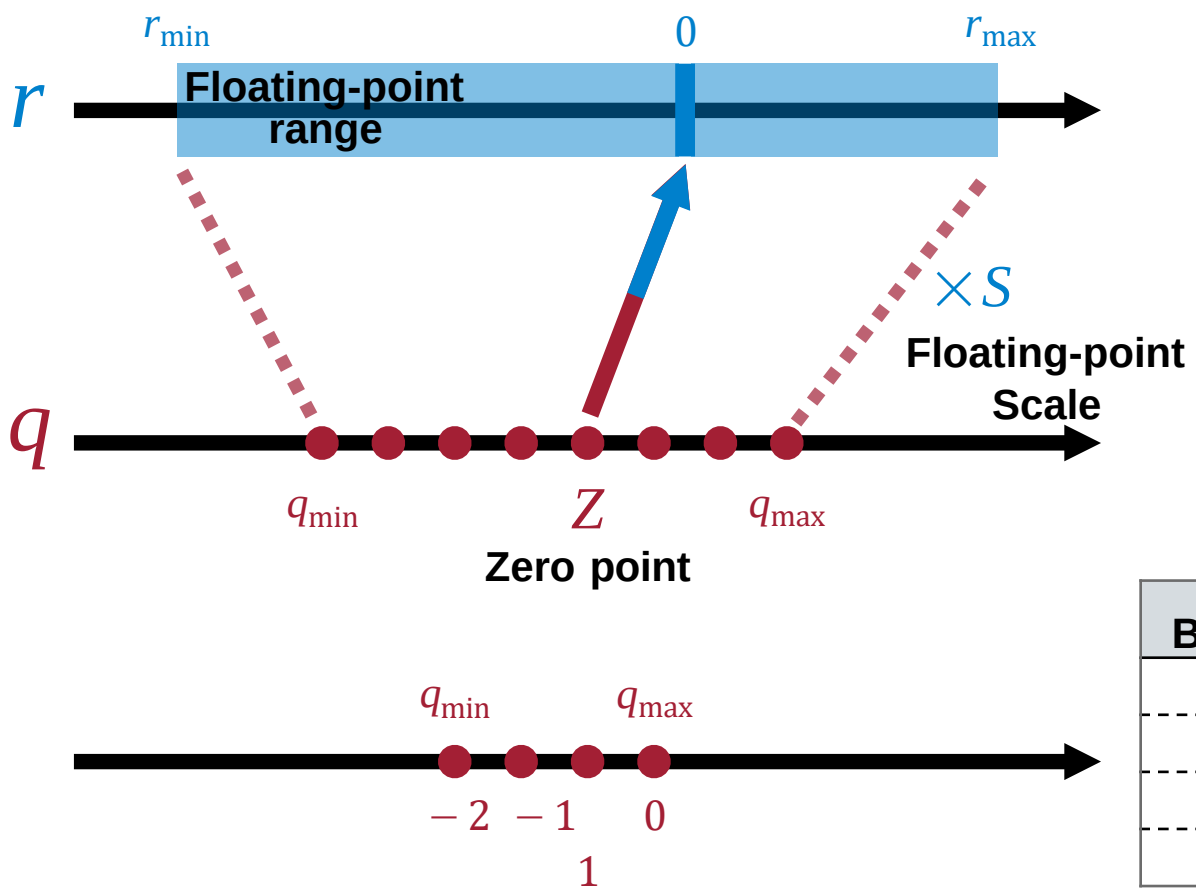
$$\downarrow$$

$$r_{\max} - r_{\min} = S\left(q_{\max} - q_{\min}\right)$$

$$S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}$$

Scale of Linear Quantization

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$



2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

Binary	Decimal
01	1
00	0
11	-1
10	-2

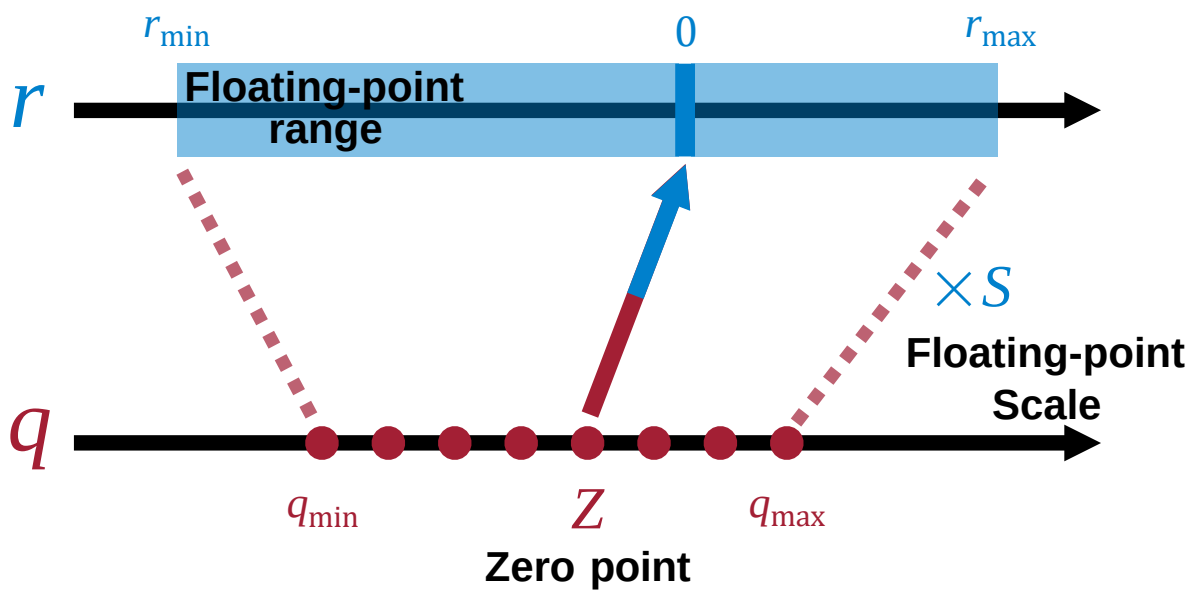
$$S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}$$

$$= \frac{2.12 - (-1.08)}{1 - (-2)}$$

$$= 1.07$$

Zero Point of Linear Quantization

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$



$$r_{\min} = S\left(q_{\min} - Z\right)$$

$$\downarrow$$

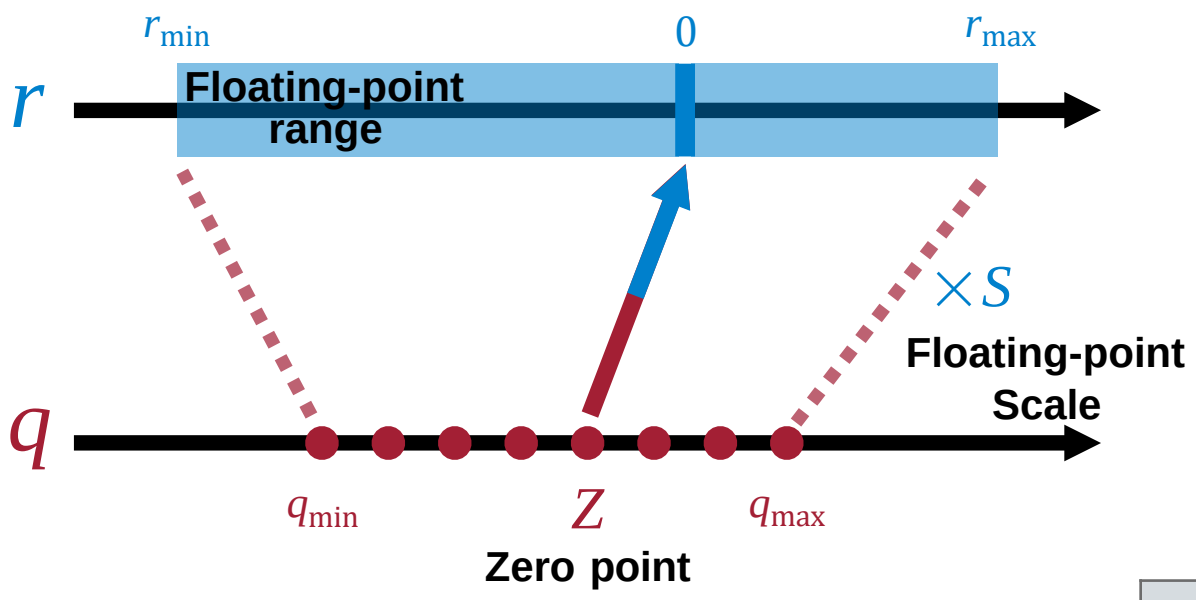
$$Z = q_{\min} - \frac{r_{\min}}{S}$$

$$\downarrow$$

$$Z = \text{round}\left(q_{\min} - \frac{r_{\min}}{S}\right)$$

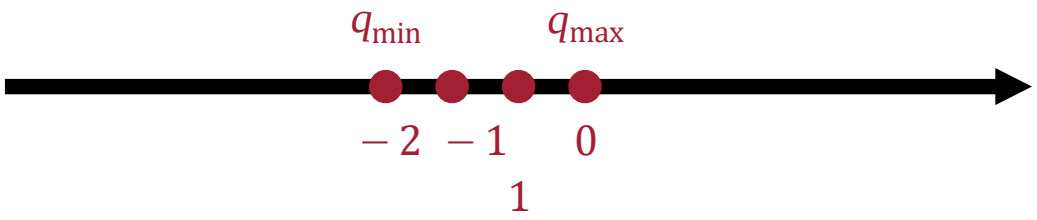
Zero Point of Linear Quantization

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$



2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

$$Z = q_{\min} - \frac{r_{\min}}{S}$$



Binary	Decimal
01	1
00	0
11	-1
10	-2

$$= \text{round}\left(-2 - \frac{-1.08}{1.07}\right)$$

$$= -1$$

TensorFlow Model Optimization Toolkit

tf.lite.Optimize

Class Variables	
DEFAULT	<Optimize.DEFAULT: 'DEFAULT'>
EXPERIMENTAL_SPARSITY	<Optimize.EXPERIMENTAL_SPARSITY: 'EXPERIMENTAL_SPARSITY'>
OPTIMIZE_FOR_LATENCY	<Optimize.OPTIMIZE_FOR_LATENCY: 'OPTIMIZE_FOR_LATENCY'>
OPTIMIZE_FOR_SIZE	<Optimize.OPTIMIZE_FOR_SIZE: 'OPTIMIZE_FOR_SIZE'>

Source: https://premioinc.com